```
/* ===== server/index.js ===== */
'use strict'

const express = require('express')
const cors = require('cors')
const helmet = require('helmet')
const config = require('./config')
const { authMiddleware } = require('./middleware/auth')
const errorHandler = require('./middleware/errorHandler')

const app = express()

// 安全响应头（HSTS / CSP / NoSniff 等）；API 无内联脚本，默认策略即可。
app.use(helmet())

// CORS 白名单：仅允许显式配置的前端来源，不再全开
app.use(cors({
  origin: config.corsOrigins,
  credentials: true
}))

app.use(express.json())

app.use('/api/auth', require('./routes/auth'))
app.use('/api/typhoons', authMiddleware, require('./routes/typhoons'))
app.use('/api/alerts', authMiddleware, require('./routes/alerts'))
app.use('/api/statistics', authMiddleware, require('./routes/statistics'))
app.use('/api/tickets', authMiddleware, require('./routes/tickets'))
app.use('/api/notifications', authMiddleware, require('./routes/notifications'))

// 无匹配路由 → 统一 JSON 404
app.use(errorHandler.notFound)

// 统一错误处理（必须在所有路由之后）
app.use(errorHandler.handler)

app.listen(config.port, () => {
  console.log(`Typhoon Warning System API running at http://localhost:${config.port}`)
})
/* ===== server/config.js ===== */
'use strict'

const fs = require('fs')
const path = require('path')
const crypto = require('crypto')

/**
 * 集中配置：密钥、端口、CORS、限流等环境相关量从 process.env 读取。
 * 无 dotenv 依赖，自带极简 .env 解析。
 */
```

```
51
52  // 极简 .env 解析：存在则读取并补全未设置的变量，缺省或解析失败不阻断启动
53  function loadEnvFile() {
54    const envPath = path.join(__dirname, '..', '.env')
55    try {
56      if (!fs.existsSync(envPath)) return
57      const text = fs.readFileSync(envPath, 'utf-8')
58      text.split('\n').forEach((line) => {
59        const trimmed = line.trim()
60        if (!trimmed || trimmed.startsWith('#')) return
61        const idx = trimmed.indexOf('=')
62        if (idx === -1) return
63        const key = trimmed.slice(0, idx).trim()
64        const value = trimmed.slice(idx + 1).trim().replace(/^["']|["']$/g, '')
65        if (process.env[key] === undefined) process.env[key] = value
66      })
67    } catch (e) {
68      console.warn('[config] 读取 .env 失败：', e.message)
69    }
70  }
71
72  loadEnvFile()
73
74  const isProd = process.env.NODE_ENV === 'production'
75
76  /**
77   * 生产环境缺少 JWT_SECRET 时直接退出，不使用默认密钥；
78   * 开发环境缺失则生成一次性随机密钥并告警，重启即失效。
79   */
80  function resolveJwtSecret() {
81    if (process.env.JWT_SECRET) return process.env.JWT_SECRET
82    if (isProd) {
83      console.error('[FATAL] 生产环境缺少 JWT_SECRET，拒绝以默认密钥启动。请在环境变量中配置。')
84      process.exit(1)
85    }
86    const ephemeral = crypto.randomBytes(32).toString('hex')
87    console.warn('[config] 未设置 JWT_SECRET，已生成一次性随机密钥（重启即失效），仅可用于本地开发。')
88    return ephemeral
89  }
90
91  function resolveCorsOrigins() {
92    const raw = process.env.CORS_ORIGINS
93    if (raw) {
94      return raw.split(',').map((s) => s.trim()).filter(Boolean)
95    }
96    return ['http://localhost:5173', 'http://127.0.0.1:5173']
97  }
98
99  module.exports = {
100   port: Number(process.env.PORT) || 3000,
```

```
  nodeEnv: process.env.NODE_ENV || 'development',
  isProd,
  // 数据目录可被 DATA_DIR 覆盖：功能测试用它隔离演示数据
  dataDir: process.env.DATA_DIR || path.join(__dirname, '..', 'data'),
  jwtSecret: resolveJwtSecret(),
  jwtExpiresIn: process.env.JWT_EXPIRES_IN || '7d',
  corsOrigins: resolveCorsOrigins(),
  loginRateLimit: {
    windowMs: Number(process.env.LOGIN_LIMIT_WINDOW_MS) || 15 * 60 * 1000,
    max: Number(process.env.LOGIN_LIMIT_MAX) || 10
  },
  registerRateLimit: {
    windowMs: Number(process.env.REGISTER_LIMIT_WINDOW_MS) || 60 * 60 * 1000,
    max: Number(process.env.REGISTER_LIMIT_MAX) || 20
  }
}
/* ===== server/lib/dataPath.js ===== */
'use strict'

/**
 * 数据文件路径入口。支持 DATA_DIR 覆盖，
 * 功能测试可用临时目录跑全链路而不污染演示数据。
 */
const path = require('path')
const config = require('../config')

function dataPath(name) {
  return path.join(config.dataDir, name)
}

module.exports = dataPath
/* ===== server/lib/jsonStore.js ===== */
'use strict'

/**
 * JSON 文件持久层，供 users / tickets / notifications 共用。
 *
 * 写锁是单进程内的一条 Promise 链。多实例或集群部署时该锁无效，
 * 需要换成数据库事务或分布式锁。
 */

const fs = require('fs')
const path = require('path')

/** 读取集合。文件不存在、为空或解析失败都返回 fallback */
function read(file, fallback = []) {
  try {
    const raw = fs.readFileSync(file, 'utf-8')
    if (!raw || !raw.trim()) return fallback
    const parsed = JSON.parse(raw)
```

```
    return Array.isArray(parsed) ? parsed : fallback
  } catch (e) {
    if (e.code === 'ENOENT') return fallback
    // 解析失败：保留现场但不要让进程挂掉
    console.warn(`[jsonStore] 读取 ${path.basename(file)} 失败，按空集合处理：`, e.message)
    return fallback
  }
}

/** 读取对象（非数组形态），语义同 read */
function readObject(file, fallback = {}) {
  try {
    const raw = fs.readFileSync(file, 'utf-8')
    if (!raw || !raw.trim()) return fallback
    const parsed = JSON.parse(raw)
    return parsed && typeof parsed === 'object' && !Array.isArray(parsed) ? parsed : fallback
  } catch (e) {
    if (e.code === 'ENOENT') return fallback
    console.warn(`[jsonStore] 读取 ${path.basename(file)} 失败，按默认对象处理：`, e.message)
    return fallback
  }
}

/** 原子写：先落 .tmp 再 rename，不直接覆盖目标文件 */
function write(file, data) {
  fs.mkdirSync(path.dirname(file), { recursive: true })
  const tmp = `${file}.tmp`
  fs.writeFileSync(tmp, JSON.stringify(data, null, 2))
  fs.renameSync(tmp, file)
}

/**
 * 全局串行写锁。
 * 关键：链条必须吞掉 rejection（then(()=>{}, ()=>{})），
 * 否则一次失败会让后续所有写操作永远挂在 rejected 的 promise 上。
 */
let chain = Promise.resolve()
function withLock(fn) {
  const run = chain.then(fn, fn)
  chain = run.then(() => {}, () => {})
  return run
}

/**
 * 读—改—写，整段在锁内。
 * 这是"并发注册不丢数据"那类正确性的来源：拆分 read 与 write 会引入 read-modify-write 竞态。
 *
 * @param {string} file
 * @param {(data:any[]) => any[]|Promise<any[]>} mutator 返回新集合（或抛错以放弃写入）
 * @param {any[]} fallback
```

```
 * @returns {Promise<any[]>} 写入后的集合
 */
async function mutate(file, mutator, fallback = []) {
  return withLock(async () => {
    const current = read(file, fallback)
    const next = await mutator(current)
    if (next === undefined || next === null) return current // 放弃写入
    write(file, next)
    return next
  })
}

async function mutateObject(file, mutator, fallback = {}) {
  return withLock(async () => {
    const current = readObject(file, fallback)
    const next = await mutator(current)
    if (next === undefined || next === null) return current
    write(file, next)
    return next
  })
}

module.exports = { read, readObject, write, withLock, mutate, mutateObject }
/* ===== server/lib/geo.js ===== */
'use strict'

/**
 * 服务端地理计算，与前端 src/utils/geo.js 互为镜像，改一侧需同步另一侧。
 * 非法输入返回 null，距离用 haversine 球面公式计算。
 */

const EARTH_RADIUS_KM = 6371

/**
 * 数值解析。null、空串、布尔值、对象与 NaN 都返回 null，
 * 其余情况转成有限数字返回。
 */
function num(v) {
  if (v === null || v === undefined) return null
  if (typeof v === 'boolean' || typeof v === 'object') return null
  const s = typeof v === 'string' ? v.trim() : v
  if (s === '') return null
  const n = Number(s)
  return Number.isFinite(n) ? n : null
}

function positiveNum(v) {
  const n = num(v)
  return n !== null && n > 0 ? n : null
}
```

```
251
252 function toRad(deg) {
253   return (deg * Math.PI) / 180
254 }
255
256 /** 球面距离（km），任一坐标非法返回 null */
257 function haversineKm(lat1, lng1, lat2, lng2) {
258   const la1 = num(lat1)
259   const ln1 = num(lng1)
260   const la2 = num(lat2)
261   const ln2 = num(lng2)
262   if (la1 === null || ln1 === null || la2 === null || ln2 === null) return null
263
264   const dLat = toRad(la2 - la1)
265   const dLng = toRad(ln2 - ln1)
266   const a =
267     Math.sin(dLat / 2) ** 2 +
268     Math.cos(toRad(la1)) * Math.cos(toRad(la2)) * Math.sin(dLng / 2) ** 2
269   const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a))
270   return EARTH_RADIUS_KM * c
271 }
272
273 /**
274  * 沿路径求最近点。只看"当前中心距离"会误导：
275  * 台风可能昨天更近（已过境）或明天更近（尚未到达）。
276  */
277 function nearestApproach(path, lat, lng) {
278   if (!Array.isArray(path) || !path.length) return null
279   const la = num(lat)
280   const ln = num(lng)
281   if (la === null || ln === null) return null
282
283   let best = null
284   path.forEach((p, i) => {
285     if (!p) return
286     const d = haversineKm(p.lat, p.lng, la, ln)
287     if (d === null) return
288     if (!best || d < best.distanceKm) best = { distanceKm: d, index: i, point: p }
289   })
290   return best
291 }
292
293 /**
294  * 风圈命中判定。风圈同心且 12 级最小，由内向外检查，第一个命中即最高级别。
295  */
296 function ringHit(distanceKm, typhoon) {
297   const d = num(distanceKm)
298   if (d === null || !typhoon) return null
299
300   const rings = [
```

```
    { key: 'radius12', label: '12 级风圈', level: 'red' },
    { key: 'radius10', label: '10 级风圈', level: 'orange' },
    { key: 'radius7', label: '7 级风圈', level: 'yellow' }
  ]
  for (const r of rings) {
    const radius = positiveNum(typhoon[r.key])
    if (radius !== null && d <= radius) {
      return { key: r.key, label: r.label, radiusKm: radius, level: r.level }
    }
  }
  return null
}

module.exports = { haversineKm, nearestApproach, ringHit }
/* ===== server/middleware/auth.js ===== */
'use strict'

const jwt = require('jsonwebtoken')
const config = require('../config')

function authMiddleware(req, res, next) {
  const header = req.headers.authorization
  if (!header || !header.startsWith('Bearer ')) {
    return res.status(401).json({ code: 401, message: '请先登录' })
  }
  const token = header.split(' ')[1]
  if (!token) {
    return res.status(401).json({ code: 401, message: '请先登录' })
  }
  try {
    const decoded = jwt.verify(token, config.jwtSecret)
    req.user = decoded
    next()
  } catch {
    return res.status(401).json({ code: 401, message: '登录已过期，请重新登录' })
  }
}

module.exports = { authMiddleware }
/* ===== server/middleware/errorHandler.js ===== */
'use strict'

/**
 * 统一错误处理中间件。
 * 捕获路由内同步/异步抛出的异常，返回统一 JSON 错误体。
 * 生产环境不向客户端泄露堆栈。
 *
 * 签名必须是 4 参数 (err, req, res, next)，且需在所有路由之后注册。
 */
function errorHandler(err, req, res, next) {
```

```
  const status = err.status || err.statusCode || 500
  if (status >= 500) {
    console.error('[server] 未处理异常: ', err && err.stack ? err.stack : err)
  }
  res.status(status).json({
    code: status,
    message: status >= 500 ? '服务器内部错误' : (err && err.message) || '请求处理失败'
  })
}

/** 无匹配的 API 路由时返回 JSON，而不是 Express 默认的 HTML */
function notFoundHandler(req, res) {
  res.status(404).json({ code: 404, message: '接口不存在' })
}

module.exports = errorHandler
module.exports.handler = errorHandler
module.exports.notFound = notFoundHandler
/* ===== server/middleware/rateLimit.js ===== */
'use strict'

const rateLimit = require('express-rate-limit')
const config = require('../config')

/**
 * 登录限流：防暴力破解。每 IP 在窗口内超过阈值返回 429 + 统一错误体。
 */
const loginLimiter = rateLimit({
  windowMs: config.loginRateLimit.windowMs,
  max: config.loginRateLimit.max,
  standardHeaders: true,
  legacyHeaders: false,
  handler: (req, res) =>
    res.status(429).json({ code: 429, message: '登录尝试过于频繁，请稍后再试' })
})

/**
 * 注册限流：防批量撞库/刷号。阈值略宽。
 */
const registerLimiter = rateLimit({
  windowMs: config.registerRateLimit.windowMs,
  max: config.registerRateLimit.max,
  standardHeaders: true,
  legacyHeaders: false,
  handler: (req, res) =>
    res.status(429).json({ code: 429, message: '注册过于频繁，请稍后再试' })
})

module.exports = { loginLimiter, registerLimiter }
/* ===== server/constants/ticket.js ===== */
```

```
'use strict'

/**
 * 工单状态机。后端路由、单元测试与前端展示共用同一套状态语义。
 */

const TICKET_STATUS = {
  PENDING: 'pending',
  PROCESSING: 'processing',
  CLOSED: 'closed'
}

const STATUS_META = {
  pending: { key: 'pending', label: '待处置', badge: 'badge-blue', order: 1 },
  processing: { key: 'processing', label: '处置中', badge: 'badge-yellow', order: 2 },
  closed: { key: 'closed', label: '已闭环', badge: 'badge-green', order: 3 }
}

/**
 * 合法状态转移表。closed 为终态，不能再改状态，但仍可追加处置意见。
 */
const ALLOWED_TRANSITIONS = {
  pending: ['processing'],
  processing: ['pending', 'closed'],
  closed: []
}

/** 时间线条目类型 */
const TIMELINE_TYPES = {
  CREATE: 'create',
  STATUS: 'status',
  COMMENT: 'comment'
}

function isStatus(value) {
  return Object.prototype.hasOwnProperty.call(STATUS_META, value)
}

/**
 * 校验状态流转是否合法（纯函数，可单测）
 * @returns {{ok:boolean, message?:string}}
 */
function canTransition(from, to) {
  if (!isStatus(to)) return { ok: false, message: `目标状态非法：${to}` }
  if (!isStatus(from)) return { ok: false, message: `当前状态非法：${from}` }
  if (from === to) return { ok: false, message: `工单已处于「${STATUS_META[to].label}」` }

  const allowed = ALLOWED_TRANSITIONS[from] || []
  if (!allowed.includes(to)) {
    const fromLabel = STATUS_META[from].label
```

```
    const toLabel = STATUS_META[to].label
    // 终态单独给一句人话提示，比"不允许"更有用
    if (from === TICKET_STATUS.CLOSED) {
      return { ok: false, message: '工单已闭环，不可再变更状态（仍可补充处置意见）' }
    }
    return { ok: false, message: `不允许从「${fromLabel}」直接变更为「${toLabel}」` }
  }
  return { ok: true }
}

function statusLabel(status) {
  return STATUS_META[status] ? STATUS_META[status].label : status || '—'
}

function statusList() {
  return Object.values(STATUS_META).sort((a, b) => a.order - b.order)
}

module.exports = {
  TICKET_STATUS,
  STATUS_META,
  ALLOWED_TRANSITIONS,
  TIMELINE_TYPES,
  isStatus,
  canTransition,
  statusLabel,
  statusList
}
/* ===== server/data/cities.js ===== */
'use strict'

/**
 * 沿海城市经纬度表（服务端版，P2-1）
 *
 * 与前端 src/constants/cities.js 同源双份：服务端要判断"城市是否进入风圈"，
 * 规则引擎必须自己有坐标。**改一侧必须同步另一侧**，否则前后端结论会打架。
 *
 * 说明：
 * 1. 坐标为城市中心近似值，用于距离测算与影响评估，精度满足百公里级判断。
 * 2. 香港、澳门按一个中国原则列为中国城市，不作为独立国家/地区。
 * 3. 未收录的城市一律返回 null —— 绝不臆造坐标。
 */

const CITY_COORDS = [
  { name: '大连', lat: 38.91, lng: 121.61 },
  { name: '天津', lat: 39.08, lng: 117.20 },
  { name: '烟台', lat: 37.54, lng: 121.40 },
  { name: '青岛', lat: 36.07, lng: 120.38 },
  { name: '连云港', lat: 34.60, lng: 119.16 },
  { name: '南通', lat: 32.02, lng: 120.89 },
```

```
  { name: '上海', lat: 31.23, lng: 121.47 },
  { name: '杭州', lat: 30.27, lng: 120.16 },
  { name: '宁波', lat: 29.87, lng: 121.55 },
  { name: '舟山', lat: 30.02, lng: 122.11 },
  { name: '台州', lat: 28.66, lng: 121.42 },
  { name: '温州', lat: 28.00, lng: 120.70 },
  { name: '福州', lat: 26.07, lng: 119.30 },
  { name: '泉州', lat: 24.87, lng: 118.68 },
  { name: '厦门', lat: 24.48, lng: 118.09 },
  { name: '漳州', lat: 24.51, lng: 117.65 },
  { name: '汕头', lat: 23.35, lng: 116.68 },
  { name: '汕尾', lat: 22.78, lng: 115.37 },
  { name: '深圳', lat: 22.54, lng: 114.06 },
  { name: '中国香港', lat: 22.32, lng: 114.17 },
  { name: '东莞', lat: 23.02, lng: 113.75 },
  { name: '广州', lat: 23.13, lng: 113.26 },
  { name: '珠海', lat: 22.27, lng: 113.55 },
  { name: '中国澳门', lat: 22.20, lng: 113.54 },
  { name: '阳江', lat: 21.86, lng: 111.98 },
  { name: '茂名', lat: 21.66, lng: 110.92 },
  { name: '湛江', lat: 21.20, lng: 110.36 },
  { name: '北海', lat: 21.47, lng: 109.12 },
  { name: '钦州', lat: 21.98, lng: 108.62 },
  { name: '防城港', lat: 21.63, lng: 108.35 },
  { name: '南宁', lat: 22.82, lng: 108.32 },
  { name: '海口', lat: 20.04, lng: 110.32 },
  { name: '三亚', lat: 18.25, lng: 109.51 }
]

/** 别名 → 标准名：数据源常写简称「香港」「澳门」，不映射会造成假阴性 */
const CITY_ALIASES = {
  香港: '中国香港',
  澳门: '中国澳门'
}

const CITY_MAP = CITY_COORDS.reduce((acc, c) => {
  acc[c.name] = c
  return acc
}, {})

function allCities() {
  return [...CITY_COORDS].sort((a, b) => b.lat - a.lat)
}

function findCity(name) {
  if (!name) return null
  const raw = String(name).trim()
  const key = CITY_ALIASES[raw] || raw
  return CITY_MAP[key] || null
}
```

```
551
552 function canonicalCityName(name) {
553   if (!name) return ''
554   const raw = String(name).trim()
555   return CITY_ALIASES[raw] || raw
556 }
557
558 module.exports = { CITY_COORDS, allCities, findCity, canonicalCityName }
559 /* ===== server/data/mock-data.js ===== */
560 const typhoons = [
561   {
562     id: 'TY202401',
563     name: '海燕',
564     nameEn: 'Haiyan',
565     status: 'active',
566     level: '超强台风',
567     windSpeed: 58,
568     pressure: 925,
569     latitude: 21.5,
570     longitude: 126.8,
571     warningLevel: 'red',
572     affectedCities: ['福州', '温州', '宁波', '上海'],
573     description: '第3号台风"海燕"正在向西北方向移动，强度持续增强，预计将在福建至浙江一带沿海登陆。',
574     startTime: '2026-05-15 08:00',
575     updateTime: '2026-05-19 14:00',
576     radius7: 380,
577     radius10: 150,
578     radius12: 80,
579     path: [
580       { lat: 15.2, lng: 135.5, time: '2026-05-15 08:00', wind: 35, level: '台风' },
581       { lat: 16.8, lng: 133.2, time: '2026-05-16 08:00', wind: 42, level: '强台风' },
582       { lat: 18.5, lng: 130.6, time: '2026-05-17 08:00', wind: 50, level: '强台风' },
583       { lat: 20.0, lng: 128.5, time: '2026-05-18 08:00', wind: 55, level: '超强台风' },
584       { lat: 21.5, lng: 126.8, time: '2026-05-19 14:00', wind: 58, level: '超强台风' },
585       { lat: 23.2, lng: 124.5, time: '2026-05-20 08:00', wind: 55, level: '超强台风' },
586       { lat: 25.0, lng: 122.0, time: '2026-05-21 08:00', wind: 48, level: '强台风' },
587       { lat: 26.5, lng: 120.5, time: '2026-05-22 08:00', wind: 40, level: '台风' }
588     ]
589   },
590   {
591     id: 'TY202402',
592     name: '山竹',
593     nameEn: 'Mangkhut',
594     status: 'active',
595     level: '强台风',
596     windSpeed: 45,
597     pressure: 955,
598     latitude: 16.2,
599     longitude: 118.5,
600     warningLevel: 'orange',
```

```
    affectedCities: ['湛江', '海口', '三亚', '北海'],
    description: '第4号台风"山竹"正在南海中部海域向西北偏西方向移动，预计将在广东西部至海南一带登陆。',
    startTime: '2026-05-17 02:00',
    updateTime: '2026-05-19 14:00',
    radius7: 320,
    radius10: 120,
    radius12: 60,
    path: [
      { lat: 12.5, lng: 125.0, time: '2026-05-17 02:00', wind: 30, level: '强热带风暴' },
      { lat: 14.0, lng: 122.3, time: '2026-05-18 02:00', wind: 38, level: '台风' },
      { lat: 15.2, lng: 120.0, time: '2026-05-19 08:00', wind: 43, level: '强台风' },
      { lat: 16.2, lng: 118.5, time: '2026-05-19 14:00', wind: 45, level: '强台风' },
      { lat: 17.5, lng: 116.0, time: '2026-05-20 08:00', wind: 42, level: '强台风' },
      { lat: 19.0, lng: 113.5, time: '2026-05-21 08:00', wind: 38, level: '台风' },
      { lat: 20.5, lng: 111.0, time: '2026-05-22 08:00', wind: 30, level: '强热带风暴' }
    ]
  },
  {
    id: 'TY202403',
    name: '利奇马',
    nameEn: 'Lekima',
    status: 'active',
    level: '热带风暴',
    windSpeed: 25,
    pressure: 992,
    latitude: 10.8,
    longitude: 142.3,
    warningLevel: 'blue',
    affectedCities: [],
    description: '第5号台风"利奇马"正在菲律宾以东洋面生成发展，目前强度为热带风暴级。',
    startTime: '2026-05-18 20:00',
    updateTime: '2026-05-19 14:00',
    radius7: 180,
    radius10: 0,
    radius12: 0,
    path: [
      { lat: 9.5, lng: 145.0, time: '2026-05-18 20:00', wind: 20, level: '热带低压' },
      { lat: 10.2, lng: 143.5, time: '2026-05-19 08:00', wind: 23, level: '热带风暴' },
      { lat: 10.8, lng: 142.3, time: '2026-05-19 14:00', wind: 25, level: '热带风暴' }
    ]
  },
  {
    id: 'TY202305',
    name: '烟花',
    nameEn: 'In-Fa',
    status: 'historical',
    level: '台风',
    windSpeed: 42,
    pressure: 960,
    latitude: 30.2,
```

```
    longitude: 122.1,
    warningLevel: 'orange',
    affectedCities: ['舟山', '宁波', '上海', '杭州'],
    description: '2025年第5号台风"烟花"已在浙江舟山登陆，造成较大风雨影响。',
    startTime: '2025-07-18 08:00',
    updateTime: '2025-07-26 20:00',
    radius7: 300,
    radius10: 100,
    radius12: 0,
    path: [
      { lat: 20.0, lng: 135.0, time: '2025-07-18 08:00', wind: 25, level: '热带风暴' },
      { lat: 22.5, lng: 132.0, time: '2025-07-20 08:00', wind: 35, level: '台风' },
      { lat: 25.0, lng: 128.5, time: '2025-07-22 08:00', wind: 42, level: '台风' },
      { lat: 27.5, lng: 125.0, time: '2025-07-24 08:00', wind: 42, level: '台风' },
      { lat: 30.2, lng: 122.1, time: '2025-07-26 08:00', wind: 35, level: '台风' },
      { lat: 32.0, lng: 119.0, time: '2025-07-26 20:00', wind: 20, level: '热带风暴' }
    ]
  },
  {
    id: 'TY202303',
    name: '杜苏芮',
    nameEn: 'Doksuri',
    status: 'historical',
    level: '超强台风',
    windSpeed: 62,
    pressure: 915,
    latitude: 24.8,
    longitude: 118.5,
    warningLevel: 'red',
    affectedCities: ['泉州', '厦门', '漳州', '福州'],
    description: '2025年第3号台风"杜苏芮"以超强台风级别在福建泉州登陆，为近十年登陆该地区最强台风。',
    startTime: '2025-07-21 08:00',
    updateTime: '2025-07-29 08:00',
    radius7: 400,
    radius10: 180,
    radius12: 100,
    path: [
      { lat: 12.0, lng: 140.0, time: '2025-07-21 08:00', wind: 20, level: '热带低压' },
      { lat: 14.5, lng: 137.0, time: '2025-07-23 08:00', wind: 35, level: '台风' },
      { lat: 17.0, lng: 133.0, time: '2025-07-25 08:00', wind: 52, level: '超强台风' },
      { lat: 20.0, lng: 128.0, time: '2025-07-27 08:00', wind: 62, level: '超强台风' },
      { lat: 22.5, lng: 123.0, time: '2025-07-28 08:00', wind: 55, level: '超强台风' },
      { lat: 24.8, lng: 118.5, time: '2025-07-29 08:00', wind: 30, level: '强热带风暴' }
    ]
  },
  {
    id: 'TY202402H',
    name: '梅花',
    nameEn: 'Muifa',
    status: 'historical',
```

```
    level: '强台风',
    windSpeed: 50,
    pressure: 940,
    latitude: 29.5,
    longitude: 122.0,
    warningLevel: 'red',
    affectedCities: ['上海', '宁波', '舟山', '南通'],
    description: '2025年第12号台风"梅花"在上海沿海擦过北上，给长三角地区带来严重风雨影响。',
    startTime: '2025-09-08 08:00',
    updateTime: '2025-09-15 08:00',
    radius7: 350,
    radius10: 140,
    radius12: 70,
    path: [
      { lat: 18.0, lng: 138.0, time: '2025-09-08 08:00', wind: 25, level: '热带风暴' },
      { lat: 21.0, lng: 133.0, time: '2025-09-10 08:00', wind: 40, level: '台风' },
      { lat: 24.0, lng: 128.0, time: '2025-09-12 08:00', wind: 50, level: '强台风' },
      { lat: 27.0, lng: 124.5, time: '2025-09-14 08:00', wind: 50, level: '强台风' },
      { lat: 29.5, lng: 122.0, time: '2025-09-15 08:00', wind: 38, level: '台风' }
    ]
  },
  {
    id: 'TY202401H',
    name: '安比',
    nameEn: 'Ampil',
    status: 'historical',
    level: '台风',
    windSpeed: 38,
    pressure: 970,
    latitude: 22.0,
    longitude: 114.0,
    warningLevel: 'yellow',
    affectedCities: ['深圳', '香港', '珠海', '广州'],
    description: '2025年第7号台风"安比"在广东深圳附近登陆，珠江口地区出现大风暴雨。',
    startTime: '2025-08-10 08:00',
    updateTime: '2025-08-16 20:00',
    radius7: 250,
    radius10: 80,
    radius12: 0,
    path: [
      { lat: 16.0, lng: 125.0, time: '2025-08-10 08:00', wind: 20, level: '热带低压' },
      { lat: 18.5, lng: 120.0, time: '2025-08-12 08:00', wind: 30, level: '强热带风暴' },
      { lat: 20.5, lng: 117.0, time: '2025-08-14 08:00', wind: 38, level: '台风' },
      { lat: 22.0, lng: 114.0, time: '2025-08-16 08:00', wind: 35, level: '台风' },
      { lat: 23.5, lng: 111.0, time: '2025-08-16 20:00', wind: 20, level: '热带风暴' }
    ]
  },
  {
    id: 'TY202302',
    name: '泰利',
```

```
    nameEn: 'Talim',
    status: 'historical',
    level: '强热带风暴',
    windSpeed: 30,
    pressure: 985,
    latitude: 21.5,
    longitude: 110.0,
    warningLevel: 'yellow',
    affectedCities: ['海口', '湛江', '北海', '南宁'],
    description: '2025年第2号台风"泰利"在海南文昌至广东徐闻一带沿海登陆。',
    startTime: '2025-06-12 08:00',
    updateTime: '2025-06-15 08:00',
    radius7: 200,
    radius10: 60,
    radius12: 0,
    path: [
      { lat: 15.0, lng: 118.0, time: '2025-06-12 08:00', wind: 20, level: '热带低压' },
      { lat: 17.0, lng: 115.0, time: '2025-06-13 08:00', wind: 28, level: '强热带风暴' },
      { lat: 19.0, lng: 112.5, time: '2025-06-14 08:00', wind: 30, level: '强热带风暴' },
      { lat: 21.5, lng: 110.0, time: '2025-06-15 08:00', wind: 25, level: '热带风暴' }
    ]
  }
]

const alerts = [
  {
    id: 'ALT001',
    typhoonId: 'TY202401',
    level: 'red',
    title: '台风红色预警',
    content: '超强台风"海燕"预计将于48小时内影响福建至浙江沿海，中心最大风力17级以上，请沿海地区做好防台防汛准备。',
    publishTime: '2026-05-19 14:00',
    city: '福州'
  },
  {
    id: 'ALT002',
    typhoonId: 'TY202401',
    level: 'red',
    title: '台风红色预警',
    content: '超强台风"海燕"逼近，温州沿海将出现6-8米巨浪，请沿海居民立即转移到安全地带。',
    publishTime: '2026-05-19 12:00',
    city: '温州'
  },
  {
    id: 'ALT003',
    typhoonId: 'TY202402',
    level: 'orange',
    title: '台风橙色预警',
    content: '强台风"山竹"正在逼近南海北部，预计未来24小时内将在广东西部至海南东部沿海登陆。',
    publishTime: '2026-05-19 14:00',
```

```
    city: '湛江'
  },
  {
    id: 'ALT004',
    typhoonId: 'TY202402',
    level: 'orange',
    title: '台风橙色预警',
    content: '强台风"山竹"预计影响海南岛，海口市将有10-12级大风，伴有暴雨到大暴雨。',
    publishTime: '2026-05-19 10:00',
    city: '海口'
  },
  {
    id: 'ALT005',
    typhoonId: 'TY202403',
    level: 'blue',
    title: '台风蓝色预警',
    content: '热带风暴"利奇马"正在菲律宾以东洋面发展，目前距离我国尚远，请持续关注最新预报。',
    publishTime: '2026-05-19 08:00',
    city: '关岛'
  },
  {
    id: 'ALT006',
    typhoonId: 'TY202401',
    level: 'yellow',
    title: '台风黄色预警',
    content: '超强台风"海燕"外围环流将影响宁波地区，预计将出现8-10级大风和暴雨。',
    publishTime: '2026-05-19 08:00',
    city: '宁波'
  },
  {
    id: 'ALT101',
    typhoonId: 'TY202305',
    level: 'red',
    title: '台风红色预警(已解除)',
    content: '台风"烟花"已在浙江舟山登陆，预计将继续向西北方向移动，强度逐渐减弱。',
    publishTime: '2025-07-25 08:00',
    city: '舟山'
  },
  {
    id: 'ALT102',
    typhoonId: 'TY202303',
    level: 'red',
    title: '台风红色预警(已解除)',
    content: '超强台风"杜苏芮"即将在泉州沿海登陆，这是近十年登陆福建的最强台风，请全力做好防御。',
    publishTime: '2025-07-28 06:00',
    city: '泉州'
  }
]

module.exports = { typhoons, alerts }
```

```
/* ===== server/services/alertRules.js ===== */
'use strict'

/**
 * 预警规则引擎，纯函数、零 I/O，输入台风与城市即可输出预警。
 *
 * 预警 ID 由（台风、城市、规则）确定生成，接口每次实时派生，
 * 工单与通知据此稳定关联到同一条预警。
 * 每条预警带 ruleKey / ruleLabel / reason；同一城市命中多条规则时
 * 只保留最严重的，其余记在 suppressed 里。
 */

const { haversineKm, nearestApproach, ringHit } = require('../lib/geo')
const { allCities, findCity, canonicalCityName } = require('../data/cities')

/** 级别序：越大越严重，用于比较与上调 */
const LEVEL_RANK = { blue: 1, yellow: 2, orange: 3, red: 4 }

const LEVEL_LABEL = {
  red: '红色预警',
  orange: '橙色预警',
  yellow: '黄色预警',
  blue: '蓝色预警'
}

/** 风圈命中规则，命中判定由 lib/geo.ringHit() 完成，这里只提供展示元数据 */
const RING_RULES = [
  { key: 'RING_12', label: '12 级风圈命中', describe: '城市到台风路径最近距离 ≤ 12 级风圈半径', level: 'red' },
  { key: 'RING_10', label: '10 级风圈命中', describe: '城市到台风路径最近距离 ≤ 10 级风圈半径', level: 'orange' },
  { key: 'RING_7', label: '7 级风圈命中', describe: '城市到台风路径最近距离 ≤ 7 级风圈半径', level: 'yellow' }
]

/** 风圈字段 → 规则 key */
const RING_KEY_BY_FIELD = { radius12: 'RING_12', radius10: 'RING_10', radius7: 'RING_7' }

/**
 * 距离/强度规则：未进风圈时按"逼近程度 + 自身强度"预警。
 * 顺序即优先级，自上而下命中即停。
 * test(ctx, typhoon) 返回命中对象或 null。
 */
const DISTANCE_RULES = [
  {
    key: 'STRONG_NEAR',
    label: '强台风逼近',
    describe: '最近距离 ≤ 300km 且中心风速 ≥ 41.5 m/s（台风级）',
    level: 'orange',
    test: (ctx, t) => (ctx.distanceKm <= 300 && Number(t.windSpeed) >= 41.5 ? {} : null)
  },
  {
    key: 'WATCH',
```

```
    label: '外围关注',
    describe: '最近距离 ≤ 500km 且中心风速 ≥ 24.5 m/s（强热带风暴级）',
    level: 'yellow',
    test: (ctx, t) => (ctx.distanceKm <= 500 && Number(t.windSpeed) >= 24.5 ? {} : null)
  }
]

/** 极低压叠加修正：不单独成条，只在命中风圈且贴近时上调一级 */
function pressureEscalation(near, typhoon) {
  const r7 = positiveNum(typhoon.radius7)
  const pressure = num(typhoon.pressure)
  // 气压缺失时不参与升级判定
  if (pressure === null) return null
  if (r7 === null) return null
  if (pressure > 950) return null
  if (near.distanceKm > r7 * 1.5) return null
  return { reason: `中心气压 ${pressure} hPa（≤950），影响程度上调一级` }
}

function num(v) {
  if (v === null || v === undefined) return null
  if (typeof v === 'boolean' || typeof v === 'object') return null
  const s = typeof v === 'string' ? v.trim() : v
  if (s === '') return null
  const n = Number(s)
  return Number.isFinite(n) ? n : null
}

/** 正数值（>0），用于"0 表示不存在"的风圈半径字段 */
function positiveNum(v) {
  const n = num(v)
  return n !== null && n > 0 ? n : null
}

/** 确定性 ID：同一 (台风, 城市, 规则) 永远得到同一个 ID */
function makeId(typhoonId, city, ruleKey) {
  return `RULE-${typhoonId}-${city}-${ruleKey}`
}

function formatKm(km) {
  return `${Math.round(km)}km`
}

/**
 * 对单个台风 × 单个城市求值，返回命中结果（未命中返回 null）
 */
function evaluateCity(typhoon, city) {
  if (!typhoon || !city) return null

  const path = Array.isArray(typhoon.path) ? typhoon.path : []
```

```
  const near = nearestApproach(path, city.lat, city.lng)
  // 无路径时退化为当前中心距离；两者都取不到则跳过（不臆造数据）
  const currentKm = haversineKm(typhoon.latitude, typhoon.longitude, city.lat, city.lng)
  const distanceKm = near ? near.distanceKm : currentKm
  if (distanceKm === null) return null

  const ring = ringHit(distanceKm, typhoon)
  const ctx = { distanceKm, ring, point: near ? near.point : null }

  // 风圈命中优先（ring 已由内向外取到最高级别）；未进圈才走距离/强度规则
  let hit = null
  if (ring) {
    hit = { ruleKey: RING_KEY_BY_FIELD[ring.key], level: ring.level }
  } else {
    for (const rule of DISTANCE_RULES) {
      if (rule.test(ctx, typhoon)) { hit = { ruleKey: rule.key, level: rule.level }; break }
    }
  }
  if (!hit) return null

  const rule = [...RING_RULES, ...DISTANCE_RULES].find((r) => r.key === hit.ruleKey)
  let level = hit.level

  // 极低压修正
  const esc = pressureEscalation(ctx, typhoon)
  if (esc && LEVEL_RANK[level] < LEVEL_RANK.red) {
    level = level === 'yellow' ? 'orange' : 'red'
  }

  const wind = num(typhoon.windSpeed)
  const reasonParts = [
    `最近距离 ${formatKm(distanceKm)}`,
    ring ? `落入${ring.label}（半径 ${Math.round(ring.radiusKm)}km）` : '未进入风圈',
    wind !== null ? `中心风速 ${wind} m/s` : null,
    esc ? esc.reason : null
  ].filter(Boolean)

  return {
    ruleKey: rule.key,
    ruleLabel: rule.label,
    ruleDescribe: rule.describe,
    level,
    distanceKm: Math.round(distanceKm * 100) / 100,
    reason: reasonParts.join('，'),
    at: ctx.point ? ctx.point.time : null,
    escalated: !!esc
  }
}

/**
```

```
 * 派生预警主入口。
 *
 * @param {object[]} typhoons 台风列表（内部只处理 status==='active'）
 * @param {object} [options]
 * @param {string[]} [options.cities] 参与计算的城市名；缺省用台风的 affectedCities + 全量表
 * @param {string} [options.now] 发布时间（便于测试固定时间）
 * @returns {object[]} 派生预警数组
 */
function deriveAlerts(typhoons, options = {}) {
  const list = Array.isArray(typhoons) ? typhoons : []
  const active = list.filter((t) => t && t.status === 'active')
  if (!active.length) return []

  const now = options.now || new Date().toISOString()
  const out = []

  active.forEach((t) => {
    // 城市范围：台风声明的影响城市（别名归一）优先，再补上全量沿海城市
    const declared = (Array.isArray(t.affectedCities) ? t.affectedCities : [])
      .map((n) => canonicalCityName(n))
      .filter(Boolean)
    const extras = Array.isArray(options.cities) && options.cities.length
      ? options.cities.map((n) => canonicalCityName(n))
      : allCities().map((c) => c.name)
    const cityNames = [...new Set([...declared, ...extras])]

    cityNames.forEach((name) => {
      const city = findCity(name)
      if (!city) return // 未收录坐标的城市不参与计算，绝不臆造距离
      const hit = evaluateCity(t, city)
      if (!hit) return

      // 降噪：非台风声明城市的"外围关注"不预警。
      // 否则一个台风会给整条海岸线每个城市都发一条黄色预警，预警中心会被噪音淹没。
      if (!declared.includes(name) && hit.ruleKey === 'WATCH') return

      out.push({
        id: makeId(t.id, city.name, hit.ruleKey),
        typhoonId: t.id,
        typhoonName: t.name,
        level: hit.level,
        levelLabel: LEVEL_LABEL[hit.level] || hit.level,
        title: `${t.name} · ${city.name}${LEVEL_LABEL[hit.level] || ''}`,
        content: `${t.name}（${t.nameEn || ''}）${hit.ruleLabel}：${hit.reason}。${hit.ruleDescribe}。`,
        city: city.name,
        distanceKm: hit.distanceKm,
        publishTime: now,
        // 来源与可解释字段：让用户知道这是机器派生而非人工研判
        source: 'rule',
        ruleKey: hit.ruleKey,
```

```
        ruleLabel: hit.ruleLabel,
        ruleDescribe: hit.ruleDescribe,
        reason: hit.reason,
        escalated: hit.escalated
      })
    })
  })

  // 同一城市只保留最严重的一条，被抑制的规则记入 suppressed
  const byCity = new Map()
  out.forEach((a) => {
    const key = `${a.typhoonId}::${a.city}`
    const prev = byCity.get(key)
    if (!prev) { byCity.set(key, a); return }
    if (LEVEL_RANK[a.level] > LEVEL_RANK[prev.level]) {
      a.suppressed = [...(prev.suppressed || []), prev.ruleKey]
      byCity.set(key, a)
    } else {
      prev.suppressed = [...(prev.suppressed || []), a.ruleKey]
    }
  })

  return [...byCity.values()].sort(
    (a, b) => LEVEL_RANK[b.level] - LEVEL_RANK[a.level] || a.distanceKm - b.distanceKm
  )
}

/** 规则清单（供前端展示"预警是怎么来的"） */
function listRules() {
  return [...RING_RULES, ...DISTANCE_RULES].map((r) => ({
    key: r.key,
    label: r.label,
    describe: r.describe,
    level: r.level
  }))
}

module.exports = { deriveAlerts, listRules, evaluateCity, LEVEL_RANK, LEVEL_LABEL }
/* ===== server/services/notify.js ===== */
'use strict'

/**
 * 消息中心服务，把业务动作翻译成该发给谁的通知。
 */

const jsonStore = require('../lib/jsonStore')
const dataPath = require('../lib/dataPath')

const NOTIFICATIONS_FILE = dataPath('notifications.json')

```

```
/** 通知类型 */
const NOTIFY_TYPES = {
  TICKET_CREATED: 'ticket_created',
  TICKET_STATUS: 'ticket_status',
  TICKET_COMMENT: 'ticket_comment',
  ALERT_DISPATCH: 'alert_dispatch'
}

const TYPE_LABEL = {
  ticket_created: '新工单',
  ticket_status: '状态变更',
  ticket_comment: '处置意见',
  alert_dispatch: '预警推送'
}

function newId() {
  return `NOT-${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 6)}`
}

/**
 * 生成通知。
 *
 * @param {object} payload
 * @param {string} payload.type NOTIFY_TYPES 之一
 * @param {string} payload.title
 * @param {string} payload.content
 * @param {string} [payload.level] 预警四色，用于前端配色
 * @param {string} [payload.refType] ticket | alert
 * @param {string} [payload.refId]
 * @param {string} [payload.actor] 触发者（自己触发的事件不会通知自己）
 * @param {string|null} [payload.userId] 接收者；null 表示全体广播
 * @returns {Promise<object|null>} 写入的通知；无接收者时返回 null
 */
async function push(payload) {
  const userId = payload.userId === undefined ? null : payload.userId
  // 自己触发的事件不通知自己
  if (userId && payload.actor && userId === payload.actor) return null

  const notification = {
    id: newId(),
    type: payload.type,
    typeLabel: TYPE_LABEL[payload.type] || payload.type,
    title: payload.title,
    content: payload.content || '',
    level: payload.level || 'blue',
    refType: payload.refType || '',
    refId: payload.refId || '',
    actor: payload.actor || '',
    userId,
    readBy: [],
```

```
    createdAt: new Date().toISOString()
  }

  await jsonStore.mutate(NOTIFICATIONS_FILE, (list) => [...list, notification])
  return notification
}

/** 非阻断版本：通知写失败只记告警，不影响业务主流程 */
async function pushSafe(payload) {
  try {
    return await push(payload)
  } catch (e) {
    console.warn('[notify] 通知写入失败，业务主流程不受影响：', e.message)
    return null
  }
}

/** 该用户能否看见这条通知（自己的 或 广播） */
function isVisible(notification, username) {
  if (!notification) return false
  return notification.userId === null || notification.userId === username
}

/** 是否已读。广播消息按人记录已读状态，用 readBy 数组 */
function isRead(notification, username) {
  return Array.isArray(notification.readBy) && notification.readBy.includes(username)
}

/** 未读数 */
function unreadCount(list, username) {
  return (list || []).filter((n) => isVisible(n, username) && !isRead(n, username)).length
}

module.exports = {
  NOTIFY_TYPES,
  TYPE_LABEL,
  NOTIFICATIONS_FILE,
  push,
  pushSafe,
  isVisible,
  isRead,
  unreadCount
}
/* ===== server/routes/auth.js ===== */
'use strict'

const express = require('express')
const bcrypt = require('bcryptjs')
const jwt = require('jsonwebtoken')
const config = require('../config')
```

```
const { authMiddleware } = require('../middleware/auth')
const { loginLimiter, registerLimiter } = require('../middleware/rateLimit')
const jsonStore = require('../lib/jsonStore')
const dataPath = require('../lib/dataPath')

const router = express.Router()
const USERS_FILE = dataPath('users.json')

// 文件不存在或解析失败时按空用户表处理
const readUsers = () => jsonStore.read(USERS_FILE, [])

function generateToken(user) {
  return jwt.sign(
    { id: user.id, username: user.username, email: user.email },
    config.jwtSecret,
    { expiresIn: config.jwtExpiresIn }
  )
}

function stripPassword(user) {
  const { password: _pw, ...rest } = user
  return rest
}

// 异步路由错误转发到统一错误处理中间件
const wrap = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next)

async function handleRegister(req, res) {
  const { username, password, email } = req.body
  if (!username || username.length < 3) {
    return res.json({ code: 400, message: '用户名至少3个字符' })
  }
  if (!password || password.length < 6) {
    return res.json({ code: 400, message: '密码至少6个字符' })
  }

  const hashedPassword = await bcrypt.hash(password, 10)
  const newUser = {
    id: Date.now().toString(36) + Math.random().toString(36).slice(2, 7),
    username,
    email: email || '',
    password: hashedPassword,
    createdAt: new Date().toISOString()
  }

  // 校验与写入都在锁内，并发注册不会产生重复用户名
  try {
    await jsonStore.mutate(USERS_FILE, (current) => {
      if (current.find((u) => u.username === username)) {
        const err = new Error('用户名已存在')
```

```
        err.status = 400
        err.conflict = true
        throw err
      }
      return [...current, newUser]
    })
  } catch (e) {
    if (e && e.conflict) return res.json({ code: 400, message: e.message })
    throw e
  }

  const token = generateToken(newUser)
  res.json({ code: 200, data: { user: stripPassword(newUser), token }, message: '注册成功' })
}

async function handleLogin(req, res) {
  const { username, password } = req.body
  if (!username || !password) {
    return res.json({ code: 400, message: '请输入用户名和密码' })
  }

  const users = readUsers()
  const user = users.find((u) => u.username === username)
  // 用户名不存在与密码错误返回同一文案
  if (!user || !(await bcrypt.compare(password, user.password))) {
    return res.json({ code: 400, message: '用户名或密码错误' })
  }

  const token = generateToken(user)
  res.json({ code: 200, data: { user: stripPassword(user), token }, message: '登录成功' })
}

router.post('/register', registerLimiter, wrap(handleRegister))
router.post('/login', loginLimiter, wrap(handleLogin))

router.get('/me', authMiddleware, (req, res) => {
  const users = readUsers()
  const user = users.find((u) => u.id === req.user.id)
  if (!user) {
    return res.json({ code: 404, message: '用户不存在' })
  }
  res.json({ code: 200, data: stripPassword(user) })
})

/** 用户候选：工单负责人下拉用，只暴露 username 与 email */
router.get('/users', authMiddleware, (req, res) => {
  const list = readUsers().map((u) => ({ id: u.id, username: u.username, email: u.email || '' }))
  res.json({ code: 200, data: list, total: list.length })
})

```

```
module.exports = router
/* ===== server/routes/typhoons.js ===== */
const express = require('express')
const router = express.Router()
const { typhoons } = require('../data/mock-data')

router.get('/', (req, res) => {
  const { status } = req.query
  let result = [...typhoons]
  if (status) {
    result = result.filter(t => t.status === status)
  }
  const list = result.map(t => {
    const { path, ...rest } = t
    if (t.status === 'active') {
      return { ...rest, path }
    }
    return { ...rest, pathLength: path.length, latestPath: path[path.length - 1] }
  })
  res.json({ code: 200, data: list, total: list.length })
})

router.get('/:id', (req, res) => {
  const typhoon = typhoons.find(t => t.id === req.params.id)
  if (!typhoon) {
    return res.status(404).json({ code: 404, message: '台风不存在' })
  }
  res.json({ code: 200, data: typhoon })
})

module.exports = router
/* ===== server/routes/statistics.js ===== */
'use strict'

const express = require('express')
const router = express.Router()
const { typhoons, alerts } = require('../data/mock-data')

// 强度等级规范顺序（与前端 src/constants/typhoon.js 保持一致）
const LEVEL_ORDER = ['热带低压', '热带风暴', '强热带风暴', '台风', '强台风', '超强台风']

// 解析 'YYYY-MM-DD HH:mm' 的年份与月份
function getYear(t) { return t.startTime ? t.startTime.slice(0, 4) : '' }
function getMonth(t) {
  const m = parseInt((t.startTime || '').slice(5, 7), 10)
  return m >= 1 && m <= 12 ? m : 0
}

router.get('/', (req, res) => {
  const activeTyphoons = typhoons.filter((t) => t.status === 'active')
```

```
  const historicalTyphoons = typhoons.filter((t) => t.status === 'historical')
  const activeAlerts = alerts.filter((a) =>
    a.typhoonId && typhoons.find((t) => t.id === a.typhoonId && t.status === 'active')
  )
  const redAlerts = activeAlerts.filter((a) => a.level === 'red')

  const affectedCities = new Set()
  typhoons.forEach((t) => (t.affectedCities || []).forEach((c) => affectedCities.add(c)))

  // 月度频次：按 startTime 的月份聚合全部台风（实时计算，不再写死）
  const monthCounts = Array.from({ length: 12 }, () => 0)
  typhoons.forEach((t) => {
    const m = getMonth(t)
    if (m) monthCounts[m - 1] += 1
  })
  const monthlyData = monthCounts.map((count, i) => ({ month: `${i + 1}月`, count }))

  // 强度分布：按规范顺序输出，缺失等级记为 0（实时计算）
  const levelCounts = {}
  typhoons.forEach((t) => {
    levelCounts[t.level] = (levelCounts[t.level] || 0) + 1
  })
  const levelDistribution = LEVEL_ORDER.map((name) => ({ name, value: levelCounts[name] || 0 }))

  // 历年趋势：按年聚合，含当年超强/强台风数（实时计算）
  const yearMap = {}
  typhoons.forEach((t) => {
    const y = getYear(t)
    if (!y) return
    if (!yearMap[y]) yearMap[y] = { year: y, total: 0, super: 0 }
    yearMap[y].total += 1
    if (t.level === '超强台风' || t.level === '强台风') yearMap[y].super += 1
  })
  const yearTrend = Object.values(yearMap).sort((a, b) => a.year.localeCompare(b.year))

  const latestYear = typhoons.reduce((max, t) => {
    const y = parseInt(getYear(t), 10)
    return Number.isNaN(y) ? max : Math.max(max, y)
  }, 0)

  res.json({
    code: 200,
    data: {
      activeCount: activeTyphoons.length,
      alertCount: activeAlerts.length,
      redAlertCount: redAlerts.length,
      affectedCityCount: affectedCities.size,
      totalHistorical: historicalTyphoons.length,
      monthlyData,
      levelDistribution,
```

```
      yearTrend,
      year: latestYear
    }
  })
})

module.exports = router
/* ===== server/routes/alerts.js ===== */
'use strict'

const express = require('express')
const router = express.Router()
const { alerts, typhoons } = require('../data/mock-data')
const { deriveAlerts, listRules } = require('../services/alertRules')
const notify = require('../services/notify')

const wrap = (fn) => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next)

/** 人工发布的预警：mock 静态数据，标注 source='manual' */
function manualAlerts() {
  return (alerts || []).map((a) => ({ ...a, source: 'manual' }))
}

router.get('/', (req, res) => {
  const { level, source, city, typhoonId } = req.query

  // 派生预警每次实时计算，ID 由（台风、城市、规则）确定性生成，
  // 工单与通知据此稳定关联。
  const derived = deriveAlerts(typhoons)

  let result = [...manualAlerts(), ...derived]

  if (level) result = result.filter((a) => a.level === level)
  if (source) result = result.filter((a) => a.source === source)
  if (city) result = result.filter((a) => a.city === city)
  if (typhoonId) result = result.filter((a) => a.typhoonId === typhoonId)

  result.sort((a, b) => new Date(b.publishTime) - new Date(a.publishTime))

  res.json({
    code: 200,
    data: result,
    total: result.length,
    // 分项计数便于前端展示"人工 N 条 / 规则派生 N 条"
    breakdown: {
      manual: manualAlerts().length,
      rule: derived.length
    }
  })
})
```

```
1451
1452  /** 规则清单：前端用于解释"这条预警是怎么来的" */
1453  router.get('/rules', (req, res) => {
1454    res.json({ code: 200, data: listRules() })
1455  })
1456
1457  /**
1458   * 预警推送：把选中的预警转成消息中心通知。
1459   * assignee 为空时为全体广播（userId=null）。
1460   */
1461  router.post('/dispatch', wrap(async (req, res) => {
1462    const { alertIds, assignee } = req.body || {}
1463    if (!Array.isArray(alertIds) || !alertIds.length) {
1464      return res.json({ code: 400, message: '请选择要推送的预警' })
1465    }
1466
1467    const operator = req.user ? req.user.username : 'unknown'
1468    const pool = [...manualAlerts(), ...deriveAlerts(typhoons)]
1469    const picked = pool.filter((a) => alertIds.includes(a.id))
1470    if (!picked.length) {
1471      return res.json({ code: 404, message: '未找到对应的预警' })
1472    }
1473
1474    let sent = 0
1475    for (const alert of picked) {
1476      const n = await notify.pushSafe({
1477        type: notify.NOTIFY_TYPES.ALERT_DISPATCH,
1478        title: `预警推送：${alert.title}`,
1479        content: `${alert.city || ''} | ${alert.content || ''}`,
1480        level: alert.level || 'blue',
1481        refType: 'alert',
1482        refId: alert.id,
1483        actor: operator,
1484        userId: assignee || null
1485      })
1486      if (n) sent += 1
1487    }
1488
1489    res.json({ code: 200, data: { sent, total: picked.length }, message: `已推送 ${sent} 条` })
1490  }))
1491
1492  module.exports = router
1493  /* ===== server/routes/tickets.js ===== */
1494  'use strict'
1495
1496  const express = require('express')
1497  const jsonStore = require('../lib/jsonStore')
1498  const dataPath = require('../lib/dataPath')
1499  const {
1500    TICKET_STATUS,
```

```
.tda-btn {
  border: 1px solid var(--border);
  border-radius: var(--radius-sm);
  background: var(--bg-card);
  color: var(--blue);
  font-size: 12px;
  padding: 4px 12px;
  cursor: pointer;
  font-family: inherit;
  transition: all var(--transition);
}
.tda-btn:hover:not(:disabled) { border-color: var(--blue); background: rgba(59, 130, 246, 0.1); }
.tda-btn:disabled { color: var(--text-muted); cursor: default; }

.tda-link { font-size: 12px; color: var(--blue); text-decoration: none; }
.tda-link:hover { text-decoration: underline; }

.pt-header {
  display: grid; grid-template-columns: 1.5fr 1.5fr 1fr 1fr;
  padding: 10px 0 10px 32px; font-size: 12px; color: var(--text-muted);
  font-weight: 600; text-transform: uppercase; letter-spacing: 0.5px;
  border-bottom: 1px solid var(--border);
}

.pt-item {
  display: grid; grid-template-columns: 1.5fr 1.5fr 1fr 1fr;
  padding: 14px 0 14px 32px; border-bottom: 1px solid rgba(255,255,255,0.03);
  font-size: 13px; align-items: center; position: relative;
  cursor: pointer; outline: none;
}
.pt-item:hover { background: rgba(255,255,255,0.03); }
.pt-item:focus-visible { box-shadow: inset 2px 0 0 var(--blue); }

.pt-current { background: rgba(59,130,246,0.04); }

.pt-dot {
  position: absolute; left: 10px; top: 50%; transform: translateY(-50%);
  width: 8px; height: 8px; border-radius: 50%; border: 2px solid var(--border); background: var(--bg-primar
y);
}

.pt-dot.active {
  border-color: var(--blue); background: var(--blue);
  box-shadow: 0 0 8px var(--blue-glow);
}

.pt-time { color: var(--text-secondary); }
.pt-pos { font-family: 'Courier New', monospace; font-size: 12px; color: var(--text-secondary); }
.pt-wind { font-weight: 600; }

.loading-state, .state-wrap {
```

```
  display: flex; flex-direction: column; align-items: center; justify-content: center;
  gap: 16px; min-height: 300px;
}
.center-row { display: flex; justify-content: center; }
</style>
/* ===== src/views/TyphoonCompare.vue ===== */
<template>
  <div class="compare-page">
    <h1 class="page-title">路径对比</h1>
    <p class="page-subtitle">选择 2–4 个台风，横向对比路径、强度与影响范围</p>

    <div v-if="typhoonStore.loading" class="loading-state">加载中...</div>
    <div v-else-if="typhoonStore.error" class="state-wrap">
      <ErrorState
        title="台风数据加载失败"
        :description="typhoonStore.error"
        @retry="typhoonStore.fetchTyphoons()"
      />
    </div>

    <template v-else>
      <!-- 选择器 -->
      <div class="glass-card picker">
        <div class="pk-header">
          <span class="section-title">选择台风</span>
          <span class="pk-count">已选 {{ selectedIds.length }} / {{ MAX_COMPARE }}</span>
          <button v-if="selectedIds.length" type="button" class="pk-clear" @click="clearAll">
            清空选择
          </button>
        </div>

        <div class="pk-chips">
          <button
            v-for="t in typhoons"
            :key="t.id"
            type="button"
            class="pk-chip"
            :class="{ 'is-selected': isSelected(t.id) }"
            :aria-pressed="isSelected(t.id)"
            :disabled="!isSelected(t.id) && isFull"
            :title="!isSelected(t.id) && isFull ? '已达上限，请先取消一个' : t.name"
            @click="toggle(t.id)"
          >
            <span class="pk-name">{{ t.name }}</span>
            <span class="badge" :class="levelBadgeClass(t.level)">{{ t.level }}</span>
            <span class="pk-status">{{ statusMeta(t.status).label }}</span>
          </button>
        </div>

        <p v-if="isFull" class="pk-hint">
```

```
9330            已达对比上限（{{ MAX_COMPARE }} 个）。超过 4 条路径叠加会导致地图与曲线不可读，请先取消一个再选。
9331          </p>
9332        </div>
9333
9334        <!-- 引导态 -->
9335        <div v-if="selected.length < 2" class="glass-card empty-guide">
9336          <p>请至少选择 <strong>2 个</strong>台风进行对比</p>
9337          <p class="eg-sub">已选 {{ selected.length }} 个</p>
9338        </div>
9339
9340        <template v-else>
9341          <!-- 指标对比 -->
9342          <div class="glass-card metric-card">
9343            <h3 class="section-title" style="margin-bottom: 16px;">指标对比</h3>
9344            <div class="mt-scroll">
9345              <table class="cmp-table">
9346                <thead>
9347                  <tr>
9348                    <th class="mt-label-col">指标</th>
9349                    <th v-for="(t, i) in selected" :key="t.id">
9350                      <i class="cm-dot" :style="{ background: compareColor(i) }"></i>{{ t.name }}
9351                    </th>
9352                  </tr>
9353                </thead>
9354                <tbody>
9355                  <tr>
9356                    <td class="mt-label-col">状态</td>
9357                    <td v-for="t in selected" :key="t.id">
9358                      <span class="badge" :class="statusMeta(t.status).badge">
9359                        {{ statusMeta(t.status).label }}
9360                      </span>
9361                    </td>
9362                  </tr>
9363                  <tr>
9364                    <td class="mt-label-col">强度等级</td>
9365                    <td v-for="t in selected" :key="t.id">
9366                      <span class="badge" :class="levelBadgeClass(t.level)">{{ t.level }}</span>
9367                    </td>
9368                  </tr>
9369                  <tr>
9370                    <td class="mt-label-col">最大风速</td>
9371                    <td v-for="t in selected" :key="t.id" :class="{ 'is-best': isBestWind(t) }">
9372                      {{ formatWind(t.windSpeed) }}
9373                    </td>
9374                  </tr>
9375                  <tr>
9376                    <td class="mt-label-col">中心气压</td>
9377                    <td v-for="t in selected" :key="t.id" :class="{ 'is-best': isBestPressure(t) }">
9378                      {{ formatPressure(t.pressure) }}
9379                    </td>
```

```
                </tr>
                <tr>
                  <td class="mt-label-col">7 级风圈</td>
                  <td v-for="t in selected" :key="t.id" :class="{ 'is-best': isBestRing(t) }">
                    {{ formatNumber(t.radius7, { unit: 'km' }) }}
                  </td>
                </tr>
                <tr>
                  <td class="mt-label-col">影响城市</td>
                  <td v-for="t in selected" :key="t.id">
                    {{ safeList(t.affectedCities).length || '—' }} 个
                  </td>
                </tr>
                <tr>
                  <td class="mt-label-col">路径点数</td>
                  <td v-for="t in selected" :key="t.id">{{ safeList(t.path).length }}</td>
                </tr>
              </tbody>
            </table>
          </div>
          <p class="mt-note">蓝色高亮表示该指标下最强项（风速最大 / 气压最低 / 风圈最大）。</p>
        </div>

        <!-- 路径叠加地图 -->
        <div style="margin-bottom: 28px;">
          <TyphoonCompareMap :typhoons="selected" />
        </div>

        <!-- 风速演变对比 -->
        <div class="glass-card" style="padding: 24px; margin-bottom: 28px;">
          <h3 class="section-title" style="margin-bottom: 16px;">风速演变对比</h3>
          <TyphoonCompareChart :typhoons="selected" />
        </div>
      </template>
    </template>
  </div>
</template>

<script setup>
import { computed, onMounted, ref } from 'vue'
import { useTyphoonStore } from '../stores/typhoon'
import {
  MAX_COMPARE,
  compareColor,
  levelBadgeClass,
  statusMeta
} from '../constants/typhoon'
import { formatWind, formatPressure, formatNumber, safeList } from '../utils/format'
import TyphoonCompareMap from '../components/TyphoonCompareMap.vue'
import TyphoonCompareChart from '../components/TyphoonCompareChart.vue'
```

```
import ErrorState from '../components/ErrorState.vue'

const typhoonStore = useTyphoonStore()
const selectedIds = ref([])

onMounted(() => {
  if (!typhoonStore.typhoons.length) {
    typhoonStore.fetchTyphoons().catch(() => {})
  }
})

const typhoons = computed(() => typhoonStore.typhoons)
const selected = computed(() =>
  selectedIds.value
    .map((id) => typhoonStore.typhoonById(id))
    .filter(Boolean)
)
const isFull = computed(() => selectedIds.value.length >= MAX_COMPARE)

function isSelected(id) {
  return selectedIds.value.includes(id)
}

function toggle(id) {
  if (isSelected(id)) {
    selectedIds.value = selectedIds.value.filter((v) => v !== id)
    return
  }
  // 达到上限时按钮已 disabled，这里再拦截一次
  if (isFull.value) return
  selectedIds.value = [...selectedIds.value, id]
}

function clearAll() {
  selectedIds.value = []
}

// 最强项：用于对比表中突出显示，省去逐格比对数字
const maxWind = computed(() =>
  selected.value.length ? Math.max(...selected.value.map((t) => Number(t.windSpeed) || 0)) : 0
)
const minPressure = computed(() => {
  const vals = selected.value
    .map((t) => Number(t.pressure))
    .filter((v) => Number.isFinite(v) && v > 0)
  return vals.length ? Math.min(...vals) : null
})
const maxRing = computed(() =>
  selected.value.length ? Math.max(...selected.value.map((t) => Number(t.radius7) || 0)) : 0
)
```

```
9480
9481 function isBestWind(t) {
9482   const v = Number(t.windSpeed) || 0
9483   return v > 0 && v === maxWind.value
9484 }
9485 function isBestPressure(t) {
9486   const v = Number(t.pressure)
9487   return Number.isFinite(v) && v > 0 && minPressure.value !== null && v === minPressure.value
9488 }
9489 function isBestRing(t) {
9490   const v = Number(t.radius7) || 0
9491   return v > 0 && v === maxRing.value
9492 }
9493 </script>
9494
9495 <style scoped>
9496 .compare-page { padding-bottom: 20px; }
9497
9498 .picker { padding: 16px 20px; margin-bottom: 20px; }
9499 .pk-header {
9500   display: flex; align-items: center; gap: 12px;
9501   margin-bottom: 12px; flex-wrap: wrap;
9502 }
9503 .pk-count {
9504   font-size: 12px; color: var(--text-muted);
9505   font-variant-numeric: tabular-nums;
9506 }
9507 .pk-clear {
9508   margin-left: auto;
9509   padding: 4px 12px;
9510   border-radius: var(--radius-sm);
9511   border: 1px solid var(--border);
9512   background: transparent;
9513   color: var(--text-secondary);
9514   font-size: 12px;
9515   cursor: pointer;
9516   transition: all var(--transition);
9517 }
9518 .pk-clear:hover { border-color: var(--red); color: #f87171; }
9519
9520 .pk-chips { display: flex; gap: 8px; flex-wrap: wrap; }
9521 .pk-chip {
9522   display: inline-flex; align-items: center; gap: 8px;
9523   padding: 6px 12px;
9524   border-radius: 999px;
9525   border: 1px solid var(--border);
9526   background: transparent;
9527   color: var(--text-secondary);
9528   font-size: 13px;
9529   cursor: pointer;
```

```
  transition: all var(--transition);
}
.pk-chip:hover:not(:disabled) { border-color: var(--blue); color: var(--text-primary); }
.pk-chip.is-selected {
  background: rgba(59, 130, 246, 0.15);
  border-color: var(--blue);
  color: var(--text-primary);
}
.pk-chip:disabled { opacity: 0.4; cursor: not-allowed; }
.pk-name { font-weight: 600; }
.pk-status { font-size: 11px; color: var(--text-muted); }

.pk-hint {
  margin: 10px 0 0;
  font-size: 12px;
  color: var(--text-muted);
  line-height: 1.5;
}

.empty-guide {
  padding: 60px 20px;
  text-align: center;
  color: var(--text-secondary);
}
.empty-guide p { margin: 0; }
.eg-sub { margin-top: 8px; font-size: 12px; color: var(--text-muted); }

.metric-card { padding: 20px; margin-bottom: 28px; }
.mt-scroll { overflow-x: auto; }
.cmp-table { width: 100%; border-collapse: collapse; min-width: 520px; }
.cmp-table th,
.cmp-table td {
  padding: 10px 12px;
  text-align: left;
  font-size: 13px;
  border-bottom: 1px solid rgba(255, 255, 255, 0.05);
}
.cmp-table th {
  font-size: 12px;
  color: var(--text-secondary);
  font-weight: 600;
}
.cmp-table th .cm-dot {
  display: inline-block;
  width: 9px; height: 9px;
  border-radius: 50%;
  margin-right: 6px;
}
.mt-label-col {
  color: var(--text-muted);
```

```
  font-size: 12px;
  white-space: nowrap;
}
.cmp-table td.is-best {
  color: var(--blue);
  font-weight: 700;
}
.mt-note {
  margin: 10px 0 0;
  font-size: 11px;
  color: var(--text-muted);
}

.state-wrap { padding: 40px 0; }
.loading-state {
  text-align: center; color: var(--text-muted); font-size: 14px; padding: 60px;
}
</style>
/* ===== src/views/AlertCenter.vue ===== */
<template>
  <div>
    <h1 class="page-title">预警中心</h1>
    <p class="page-subtitle">
      台风预警信息管理与追踪
      <span class="ac-breakdown">
        · 人工发布 {{ alertStore.breakdown.manual }} 条 / 规则派生 {{ alertStore.breakdown.rule }} 条
      </span>
    </p>

    <div v-if="alertStore.loading" class="loading-state">加载中...</div>
    <div v-else-if="alertStore.error" class="state-wrap">
      <ErrorState
        :title="'预警加载失败'"
        :description="alertStore.error"
        :loading="alertStore.loading"
        @retry="alertStore.fetchAlerts()"
      />
    </div>

    <template v-else>
      <!-- 地理围栏告警：基于关注城市 × 活跃台风前端派生，与下方后端预警分区展示，
           避免用户误以为是服务端新增的预警条目 -->
      <div class="ac-fence">
        <!-- 预警中心没有地图：点击城市时跳到监控页并带 ?focus= 深链定位 -->
        <FenceAlerts :typhoons="typhoonStore.typhoons" @locate="locateCity" />
      </div>

      <!-- 来源筛选：自动派生的结论必须与人工研判可区分 -->
      <div class="ac-source-row">
        <button
```

```
          class="filter-tab"
          v-for="s in SOURCE_FILTERS"
          :key="s.key"
          :class="{ active: activeSource === s.key }"
          @click="setSource(s.key)"
        >{{ s.label }} ({{ sourceCount(s.key) }})</button>
      </div>

      <!-- Filter Tabs -->
      <div class="filter-tabs" style="margin-bottom: 20px;">
        <button class="filter-tab" :class="{ active: activeFilter === '' }" @click="setFilter('')">
          全部 ({{ totalCount }})
        </button>
        <button class="filter-tab" :class="{ active: activeFilter === 'red' }" @click="setFilter('red')">
          红色预警 ({{ alertStore.redAlerts.length }})
        </button>
        <button class="filter-tab" :class="{ active: activeFilter === 'orange' }" @click="setFilter('orang
e')">
          橙色预警 ({{ alertStore.orangeAlerts.length }})
        </button>
        <button class="filter-tab" :class="{ active: activeFilter === 'yellow' }" @click="setFilter('yello
w')">
          黄色预警 ({{ alertStore.yellowAlerts.length }})
        </button>
        <button class="filter-tab" :class="{ active: activeFilter === 'blue' }" @click="setFilter('blue')">
          蓝色预警 ({{ alertStore.blueAlerts.length }})
        </button>
      </div>

      <!-- 批量推送工具条：选中后才出现，避免常驻干扰阅读 -->
      <div class="ac-toolbar glass-card" v-if="selectedIds.length">
        <span class="act-count">已选 {{ selectedIds.length }} 条</span>
        <select class="act-select" v-model="dispatchAssignee">
          <option value="">全体广播</option>
          <option v-for="u in users" :key="u.id || u.username" :value="u.username">{{ u.username }}</option>
        </select>
        <button class="btn act-btn" @click="dispatch" :disabled="dispatching">
          {{ dispatching ? '推送中...' : '推送通知' }}
        </button>
        <button class="btn act-btn" @click="selectedIds = []">清空选择</button>
      </div>

      <!-- Alert Grid -->
      <div class="alert-grid">
        <div class="alert-card glass-card fade-in" v-for="(a, i) in filteredAlerts"
             :key="a.id" :class="[`alert-card-${a.level}`, `fade-in-${Math.min(i % 4 + 1, 4)}`]">
          <div class="ac-header">
            <div class="ac-tags">
              <span class="warn-tag is-lg" :class="`warn-${a.level}`">{{ warningLabel(a.level) }}</span>
              <span class="badge" :class="alertSourceMeta(a.source).badge">{{ alertSourceMeta(a.source).label
}}</span>
              <input class="ac-check" type="checkbox" :value="a.id" v-model="selectedIds"
                     :aria-label="`选择预警 ${a.title}`" />
```

```
            </div>
            <span class="ac-time">{{ formatDateTime(a.publishTime) }}</span>
          </div>
          <h3 class="ac-title">{{ a.title }}</h3>
          <p class="ac-content">{{ a.content }}</p>
          <div class="ac-rule" v-if="a.source === 'rule' && a.ruleKey">
            命中规则：<code>{{ a.ruleKey }}</code>
          </div>
          <div class="ac-footer">
            <div class="ac-city">
              <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.5">
                <path d="M21 10c0 7-9 13-9 13s-9-6-9-13a9 9 0 0 0 1 18 0z"/>
                <circle cx="12" cy="10" r="3"/>
              </svg>
              {{ a.city || '—' }}
            </div>
            <div class="ac-actions">
              <!-- 已建单则改为跳转，避免用户重复点击撞上后端 400 -->
              <router-link v-if="ticketStore.ticketedAlertIds.has(a.id)" to="/tickets" class="ac-link">
                查看工单 →
              </router-link>
              <button v-else class="ac-btn" @click="openCreate(a)" :disabled="ticketStore.submitting">
                生成工单
              </button>
              <router-link :to="`/typhoon/${a.typhoonId}`" class="ac-link">台风详情 →</router-link>
            </div>
          </div>
        </div>
      </div>

      <div v-if="!filteredAlerts.length" class="empty-state" style="padding: 60px;">
        暂无匹配的预警信息
      </div>
    </template>

    <TicketCreateDialog
      :open="dialogOpen"
      :alert="currentAlert"
      :users="users"
      @close="dialogOpen = false"
      @created="onCreated"
    />
  </div>
</template>

<script setup>
import { ref, computed, onMounted } from 'vue'
import { useRouter } from 'vue-router'
import { useAlertStore } from '../stores/alert'
import { useTyphoonStore } from '../stores/typhoon'
```

```
import { useTicketStore } from '../stores/ticket'
import { warningLabel } from '../constants/typhoon'
import { alertSourceMeta } from '../constants/ticket'
import { formatDateTime } from '../utils/format'
import http from '../utils/request'
import { toast } from '../utils/toast'
import ErrorState from '../components/ErrorState.vue'
import FenceAlerts from '../components/FenceAlerts.vue'
import TicketCreateDialog from '../components/TicketCreateDialog.vue'

const SOURCE_FILTERS = [
  { key: '', label: '全部来源' },
  { key: 'manual', label: '人工发布' },
  { key: 'rule', label: '规则派生' }
]

const alertStore = useAlertStore()
const typhoonStore = useTyphoonStore()
const ticketStore = useTicketStore()
const router = useRouter()

/**
 * 本页无地图，点击城市不能"点了没反应"：跳监控页并用 ?focus= 深链定位。
 * 坐标是否收录由监控页判断并提示，这里不重复校验。
 */
function locateCity(city) {
  if (!city?.name) return
  router.push({ path: '/monitor', query: { focus: city.name } })
}

const activeFilter = ref('')
const activeSource = ref('')
const selectedIds = ref([])
const users = ref([])
const dispatchAssignee = ref('')
const dispatching = ref(false)
const dialogOpen = ref(false)
const currentAlert = ref(null)

// 围栏计算依赖台风数据；预警中心是独立入口，可能未加载过台风列表
// 工单列表用于判断"该预警是否已建单"；用户列表用于负责人下拉
onMounted(() => {
  if (!typhoonStore.typhoons.length) typhoonStore.fetchTyphoons()
  ticketStore.fetchTickets().catch(() => {})
  http.get('/auth/users').then((list) => { users.value = Array.isArray(list) ? list : [] }).catch(() => {})
})

const filteredAlerts = computed(() => {
  let list = alertStore.alerts
  if (activeSource.value) list = list.filter((a) => a.source === activeSource.value)
```

```
  if (activeFilter.value) return list.filter((a) => a.level === activeFilter.value)
  return list
})

const totalCount = computed(() => alertStore.alerts.length)

function sourceCount(key) {
  return key ? alertStore.groupedBySource[key]?.length || 0 : totalCount.value
}

function setFilter(level) { activeFilter.value = level }

function setSource(key) { activeSource.value = key }

function openCreate(alert) {
  currentAlert.value = alert
  dialogOpen.value = true
}

function onCreated() {
  selectedIds.value = selectedIds.value.filter((id) => id !== currentAlert.value?.id)
}

async function dispatch() {
  const ids = [...selectedIds.value]
  if (!ids.length) return
  dispatching.value = true
  try {
    const res = await alertStore.dispatch(ids, dispatchAssignee.value)
    toast.success(`已推送 ${res.sent}/${res.total} 条`)
    selectedIds.value = []
  } catch (e) {
    // 请求层已 toast，这里只复位按钮状态
    console.warn('[alerts] 推送失败: ', e)
  } finally {
    dispatching.value = false
  }
}
</script>

<style scoped>
.ac-fence { margin-bottom: 24px; }

.ac-breakdown { color: var(--text-muted); }

.ac-source-row {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
  margin-bottom: 12px;
```

```
}

.ac-toolbar {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 12px 16px;
  margin-bottom: 16px;
}

.act-count { font-size: 13px; color: var(--text-secondary); }

.act-select {
  padding: 6px 10px;
  border-radius: var(--radius-sm);
  border: 1px solid var(--border);
  background: var(--bg-primary);
  color: var(--text-primary);
  font-size: 13px;
  font-family: inherit;
  outline: none;
}
.act-select:focus { border-color: var(--blue); }

.act-btn { padding: 6px 14px; font-size: 13px; }

.alert-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(380px, 1fr));
  gap: 16px;
}

.alert-card {
  padding: 20px;
  border-left: 3px solid transparent;
}

.alert-card-red { border-left-color: var(--red); }
.alert-card-orange { border-left-color: var(--orange); }
.alert-card-yellow { border-left-color: var(--yellow); }
.alert-card-blue { border-left-color: var(--blue); }

.ac-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 10px;
}

.ac-tags { display: flex; align-items: center; gap: 8px; }
```

```
9880
9881 .ac-check { width: 14px; height: 14px; accent-color: var(--blue); cursor: pointer; }
9882
9883 .ac-time {
9884   font-size: 12px;
9885   color: var(--text-muted);
9886 }
9887
9888 .ac-title {
9889   font-size: 16px;
9890   font-weight: 600;
9891   margin-bottom: 8px;
9892 }
9893
9894 .ac-content {
9895   font-size: 13px;
9896   color: var(--text-secondary);
9897   line-height: 1.6;
9898   margin-bottom: 14px;
9899 }
9900
9901 .ac-rule {
9902   font-size: 12px;
9903   color: var(--text-muted);
9904   margin-bottom: 12px;
9905 }
9906 .ac-rule code {
9907   padding: 1px 6px;
9908   border-radius: 4px;
9909   background: rgba(255, 255, 255, 0.06);
9910   font-size: 11px;
9911 }
9912
9913 .ac-footer {
9914   display: flex;
9915   justify-content: space-between;
9916   align-items: center;
9917   gap: 12px;
9918 }
9919
9920 .ac-city {
9921   display: flex;
9922   align-items: center;
9923   gap: 6px;
9924   font-size: 13px;
9925   color: var(--text-secondary);
9926 }
9927
9928 .ac-actions { display: flex; align-items: center; gap: 12px; }
9929
```

```
.ac-btn {
  border: 1px solid var(--border);
  border-radius: var(--radius-sm);
  background: var(--bg-card);
  color: var(--blue);
  font-size: 12px;
  padding: 4px 12px;
  cursor: pointer;
  font-family: inherit;
  transition: all var(--transition);
}
.ac-btn:hover:not(:disabled) { border-color: var(--blue); background: rgba(59, 130, 246, 0.1); }
.ac-btn:disabled { color: var(--text-muted); cursor: default; }

.ac-link {
  font-size: 13px;
  color: var(--blue);
  text-decoration: none;
}

.ac-link:hover { text-decoration: underline; }

.empty-state, .loading-state, .state-wrap {
  text-align: center;
  color: var(--text-muted);
  font-size: 14px;
}

.state-wrap { padding: 40px 0; }
</style>
/* ===== src/views/TicketCenter.vue ===== */
<template>
  <div>
    <div class="tc-head">
      <div>
        <h1 class="page-title">处置工单</h1>
        <p class="page-subtitle">预警 → 工单 → 处置 → 闭环 全流程追踪</p>
      </div>
      <button class="btn" @click="refresh" :disabled="ticketStore.loading">
        {{ ticketStore.loading ? '刷新中...' : '刷新' }}
      </button>
    </div>

    <!-- 统计：以 /tickets/stats 为准，不受当前筛选影响 -->
    <div class="tc-stats" v-if="stats">
      <div class="tc-stat glass-card">
        <div class="tcs-label">工单总数</div>
        <div class="tcs-value">{{ stats.total }}</div>
      </div>
      <div class="tc-stat glass-card" v-for="s in statusList()" :key="s.key">
```

```
9980          <div class="tcs-label">{{ s.label }}</div>
9981          <div class="tcs-value">{{ stats.byStatus?.[s.key] ?? 0 }}</div>
9982        </div>
9983        <div class="tc-stat glass-card">
9984          <div class="tcs-label">待认领</div>
9985          <div class="tcs-value" :class="{ 'tcs-warn': stats.unassigned > 0 }">{{ stats.unassigned }}</div>
9986        </div>
9987        <div class="tc-stat glass-card">
9988          <div class="tcs-label">闭环率</div>
9989          <div class="tcs-value">{{ stats.closedRate }}%</div>
9990        </div>
9991      </div>
9992
9993      <div v-if="ticketStore.error" class="state-wrap">
9994        <ErrorState title="工单加载失败" :description="ticketStore.error" @retry="refresh" />
9995      </div>
9996
9997      <template v-else>
9998        <div class="filter-tabs" style="margin-bottom: 20px;">
9999          <button class="filter-tab" :class="{ active: !ticketStore.filters.status }" @click="setStatus('')">
10000           全部 ({{ ticketStore.tickets.length }})
10001         </button>
10002         <button
10003           class="filter-tab"
10004           v-for="s in statusList()"
10005           :key="s.key"
10006           :class="{ active: ticketStore.filters.status === s.key }"
10007           @click="setStatus(s.key)"
10008         >
10009           {{ s.label }} ({{ stats?.byStatus?.[s.key] ?? 0 }})
10010         </button>
10011       </div>
10012
10013       <div v-if="ticketStore.loading && !ticketStore.tickets.length" class="empty-state" style="padding: 60p
x;">
10014         加载中...
10015       </div>
10016
10017       <div v-else-if="!ticketStore.tickets.length" class="empty-state" style="padding: 60px;">
10018         暂无工单，可在预警中心一键生成
10019       </div>
10020
10021       <div class="tc-list" v-else>
10022         <div
10023           class="tc-card glass-card warn-bar"
10024           :class="`warn-${t.level || 'blue'}`"
10025           v-for="t in ticketStore.tickets"
10026           :key="t.id"
10027         >
10028           <div class="tcc-main" @click="toggle(t.id)" role="button" tabindex="0"
10029                @keydown.enter.prevent="toggle(t.id)">
```

```
10030             <div class="tcc-left">
10031               <div class="tcc-title-row">
10032                 <span class="warn-tag" :class="`warn-${t.level || 'blue'}`">{{ warningLabel(t.level) }}</span
      >
10033                 <span class="badge" :class="statusBadge(t.status)">{{ statusLabel(t.status) }}</span>
10034                 <span class="tcc-title">{{ t.title }}</span>
10035               </div>
10036               <div class="tcc-meta">
10037                 <span>{{ t.city || '未指定城市' }}</span>
10038                 <span class="tcc-dot">·</span>
10039                 <span>负责人：{{ t.assignee || '待认领' }}</span>
10040                 <span class="tcc-dot">·</span>
10041                 <span>创建：{{ t.createdBy }}</span>
10042                 <span class="tcc-dot">·</span>
10043                 <span>{{ formatDateTime(t.updatedAt) }}</span>
10044               </div>
10045             </div>
10046             <div class="tcc-right">
10047               <router-link v-if="t.typhoonId" :to="`/typhoon/${t.typhoonId}`" class="tcc-link"
10048                            @click.stop>台风详情</router-link>
10049               <span class="tcc-caret" :class="{ 'is-open': expandedId === t.id }">▾</span>
10050             </div>
10051           </div>
10052
10053           <div class="tcc-detail" v-if="expandedId === t.id">
10054             <div class="tcc-desc" v-if="t.description">{{ t.description }}</div>
10055
10056             <div class="tcc-section">
10057               <div class="tcc-section-title">处置记录</div>
10058               <TicketTimeline :timeline="t.timeline || []" />
10059             </div>
10060
10061             <div class="tcc-section">
10062               <div class="tcc-section-title">状态流转</div>
10063               <!-- 只渲染状态机允许的目标：非法目标不出现，而不是点了再报错 -->
10064               <div class="tcc-actions" v-if="nextStatuses(t.status).length">
10065                 <button
10066                   class="btn tcc-btn"
10067                   v-for="s in nextStatuses(t.status)"
10068                   :key="s"
10069                   :disabled="ticketStore.submitting"
10070                   @click="changeStatus(t, s)"
10071                 >
10072                   流转到「{{ statusLabel(s) }}」
10073                 </button>
10074               </div>
10075               <div v-else class="tcc-hint">工单已闭环，不可再变更状态（仍可补充处置意见）</div>
10076             </div>
10077
10078             <div class="tcc-section">
10079               <div class="tcc-section-title">补充处置意见</div>
```

```
              <div class="tcc-comment">
                <textarea
                  class="tcc-textarea"
                  v-model="commentDrafts[t.id]"
                  placeholder="记录处置进展、协调事项或复盘结论"
                ></textarea>
                <button
                  class="btn tcc-btn"
                  :disabled="ticketStore.submitting || !(commentDrafts[t.id] || '').trim()"
                  @click="submitComment(t)"
                >提交</button>
              </div>
            </div>
          </div>
        </div>
      </div>
    </template>
  </div>
</template>

<script setup>
import { ref, computed, reactive, onMounted } from 'vue'
import { useTicketStore } from '../stores/ticket'
import { statusList, statusLabel, statusBadge, nextStatuses } from '../constants/ticket'
import { warningLabel } from '../constants/typhoon'
import { formatDateTime } from '../utils/format'
import { toast } from '../utils/toast'
import ErrorState from '../components/ErrorState.vue'
import TicketTimeline from '../components/TicketTimeline.vue'

const ticketStore = useTicketStore()
const expandedId = ref('')
const commentDrafts = reactive({})

const stats = computed(() => ticketStore.stats)

async function refresh() {
  try {
    await Promise.all([ticketStore.fetchTickets(), ticketStore.fetchStats()])
  } catch {
    /* 错误文案由 store 记录并展示 */
  }
}

async function setStatus(status) {
  ticketStore.setFilter('status', status)
  try { await ticketStore.fetchTickets() } catch { /* noop */ }
}

function toggle(id) {
```

```
  expandedId.value = expandedId.value === id ? '' : id
}

async function changeStatus(ticket, status) {
  try {
    await ticketStore.changeStatus(ticket.id, status)
    toast.success(`已流转到「${statusLabel(status)}」`)
  } catch {
    /* 请求层已提示 */
  }
}

async function submitComment(ticket) {
  const content = (commentDrafts[ticket.id] || '').trim()
  if (!content) return
  try {
    await ticketStore.addComment(ticket.id, content)
    commentDrafts[ticket.id] = ''
    toast.success('已记录处置意见')
  } catch {
    /* 请求层已提示 */
  }
}

onMounted(refresh)
</script>

<style scoped>
.tc-head {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 16px;
}

.tc-stats {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(120px, 1fr));
  gap: 14px;
  margin-bottom: 24px;
}

.tc-stat { padding: 16px 18px; }
.tcs-label { font-size: 12px; color: var(--text-muted); margin-bottom: 6px; }
.tcs-value { font-size: 22px; font-weight: 700; font-variant-numeric: tabular-nums; }
.tcs-warn { color: #fbbf24; }

.tc-list { display: flex; flex-direction: column; gap: 12px; }

.tc-card { padding: 18px 20px; }
```

```
10180
10181 .tcc-main {
10182   display: flex;
10183   justify-content: space-between;
10184   align-items: flex-start;
10185   gap: 16px;
10186   cursor: pointer;
10187   outline: none;
10188 }
10189
10190 .tcc-title-row { display: flex; align-items: center; gap: 8px; flex-wrap: wrap; }
10191 .tcc-title { font-size: 15px; font-weight: 600; }
10192
10193 .tcc-meta {
10194   font-size: 12px;
10195   color: var(--text-muted);
10196   margin-top: 6px;
10197   display: flex;
10198   flex-wrap: wrap;
10199   gap: 4px;
10200 }
10201 .tcc-dot { opacity: 0.5; }
10202
10203 .tcc-right { display: flex; align-items: center; gap: 10px; }
10204 .tcc-link { font-size: 12px; color: var(--blue); text-decoration: none; white-space: nowrap; }
10205 .tcc-link:hover { text-decoration: underline; }
10206
10207 .tcc-caret { color: var(--text-muted); font-size: 12px; transition: transform var(--transition); }
10208 .tcc-caret.is-open { transform: rotate(180deg); }
10209
10210 .tcc-detail {
10211   margin-top: 16px;
10212   padding-top: 16px;
10213   border-top: 1px solid var(--border);
10214 }
10215
10216 .tcc-desc {
10217   font-size: 13px;
10218   color: var(--text-secondary);
10219   margin-bottom: 16px;
10220   line-height: 1.7;
10221 }
10222
10223 .tcc-section { margin-bottom: 18px; }
10224 .tcc-section:last-child { margin-bottom: 0; }
10225
10226 .tcc-section-title {
10227   font-size: 12px;
10228   color: var(--text-muted);
10229   margin-bottom: 10px;
```

```
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

.tcc-actions { display: flex; gap: 10px; flex-wrap: wrap; }

.tcc-btn { padding: 6px 14px; font-size: 13px; }

.tcc-hint { font-size: 12px; color: var(--text-muted); }

.tcc-comment { display: flex; gap: 10px; align-items: flex-start; }

.tcc-textarea {
  flex: 1;
  min-height: 62px;
  padding: 8px 12px;
  border-radius: var(--radius-sm);
  border: 1px solid var(--border);
  background: var(--bg-primary);
  color: var(--text-primary);
  font-size: 13px;
  font-family: inherit;
  line-height: 1.6;
  resize: vertical;
  outline: none;
}
.tcc-textarea:focus { border-color: var(--blue); }

.empty-state, .state-wrap {
  text-align: center;
  color: var(--text-muted);
  font-size: 14px;
}
</style>
/* ===== src/views/MessageCenter.vue ===== */
<template>
  <div>
    <div class="mc-head">
      <div>
        <h1 class="page-title">消息中心</h1>
        <p class="page-subtitle">
          工单动态与预警推送
          <span v-if="store.unread > 0" class="mc-unread">· {{ store.unread }} 条未读</span>
        </p>
      </div>
      <div class="mc-actions">
        <button class="btn" @click="refresh" :disabled="store.loading">
          {{ store.loading ? '刷新中...' : '刷新' }}
        </button>
        <button class="btn btn-primary" @click="readAll" :disabled="!store.unread">全部已读</button>
```

```
      </div>
    </div>

    <div v-if="store.error" class="state-wrap">
      <ErrorState title="消息加载失败" :description="store.error" @retry="refresh" />
    </div>

    <template v-else>
      <div class="filter-tabs" style="margin-bottom: 20px;">
        <button
          class="filter-tab"
          v-for="f in FILTERS"
          :key="f.key"
          :class="{ active: activeFilter === f.key }"
          @click="activeFilter = f.key"
        >{{ f.label }} ({{ countOf(f.key) }})</button>
      </div>

      <div v-if="store.loading && !store.notifications.length" class="empty-state" style="padding: 60px;">
        加载中...
      </div>

      <div v-else-if="!filtered.length" class="empty-state" style="padding: 60px;">
        暂无消息
      </div>

      <div class="mc-list" v-else>
        <div
          class="mc-item glass-card"
          v-for="n in filtered"
          :key="n.id"
          :class="{ 'mc-unread-item': !n.read }"
          role="button" tabindex="0"
          @click="openItem(n)"
          @keydown.enter.prevent="openItem(n)"
        >
          <div class="mci-head">
            <span class="mci-dot" v-if="!n.read"></span>
            <span class="badge" :class="notifyTypeMeta(n.type).badge">{{ notifyTypeMeta(n.type).label }}</spa
n>
            <span class="warn-tag" :class="`warn-${n.level || 'blue'}`">{{ warningLabel(n.level) }}</span>
            <span class="mci-title">{{ n.title }}</span>
            <span class="mci-time">{{ formatDateTime(n.createdAt) }}</span>
          </div>
          <div class="mci-content">{{ n.content }}</div>
          <div class="mci-foot">
            <span>来自 {{ n.actor || '系统' }}</span>
            <span v-if="n.userId === null" class="mci-broadcast">全体广播</span>
          </div>
        </div>
      </div>
```

```
    </template>
  </div>
</template>

<script setup>
import { ref, computed, onMounted } from 'vue'
import { useRouter } from 'vue-router'
import { useNotificationStore } from '../stores/notification'
import { notifyTypeMeta } from '../constants/ticket'
import { warningLabel } from '../constants/typhoon'
import { formatDateTime } from '../utils/format'
import { toast } from '../utils/toast'
import ErrorState from '../components/ErrorState.vue'

const FILTERS = [
  { key: 'all', label: '全部' },
  { key: 'unread', label: '未读' },
  { key: 'read', label: '已读' }
]

const store = useNotificationStore()
const router = useRouter()
const activeFilter = ref('all')

const filtered = computed(() => {
  if (activeFilter.value === 'unread') return store.notifications.filter((n) => !n.read)
  if (activeFilter.value === 'read') return store.notifications.filter((n) => n.read)
  return store.notifications
})

function countOf(key) {
  if (key === 'unread') return store.notifications.filter((n) => !n.read).length
  if (key === 'read') return store.notifications.filter((n) => n.read).length
  return store.notifications.length
}

async function refresh() {
  try { await store.fetchNotifications() } catch { /* store 已记录错误 */ }
}

async function readAll() {
  const n = await store.markAllRead()
  toast.success(n ? `已将 ${n} 条标记为已读` : '没有未读消息')
}

async function openItem(item) {
  if (!item.read) await store.markRead(item.id)
  if (item.refType === 'ticket') router.push('/tickets')
  else if (item.refType === 'alert') router.push('/alerts')
}
```

```
10380
10381 onMounted(() => {
10382   // 进入页面即拉一次列表
10383   refresh()
10384 })
10385 </script>
10386
10387 <style scoped>
10388 .mc-head {
10389   display: flex;
10390   align-items: flex-start;
10391   justify-content: space-between;
10392   gap: 16px;
10393 }
10394
10395 .mc-unread { color: #f87171; }
10396
10397 .mc-actions { display: flex; gap: 10px; }
10398
10399 .mc-list { display: flex; flex-direction: column; gap: 10px; }
10400
10401 .mc-item {
10402   padding: 14px 18px;
10403   cursor: pointer;
10404   border-left: 3px solid transparent;
10405   outline: none;
10406 }
10407
10408 .mc-unread-item { border-left-color: var(--blue); background: rgba(59, 130, 246, 0.06); }
10409
10410 .mci-head { display: flex; align-items: center; gap: 8px; flex-wrap: wrap; }
10411
10412 .mci-dot {
10413   width: 7px;
10414   height: 7px;
10415   border-radius: 50%;
10416   background: var(--blue);
10417   box-shadow: 0 0 8px var(--blue-glow);
10418 }
10419
10420 .mci-title { font-size: 14px; font-weight: 600; }
10421 .mci-time { margin-left: auto; font-size: 11px; color: var(--text-muted); }
10422
10423 .mci-content {
10424   font-size: 13px;
10425   color: var(--text-secondary);
10426   margin-top: 6px;
10427   line-height: 1.6;
10428 }
10429
```

```
.mci-foot {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-top: 8px;
  font-size: 11px;
  color: var(--text-muted);
}

.mci-broadcast {
  padding: 1px 7px;
  border-radius: 10px;
  background: rgba(139, 92, 246, 0.15);
  color: #a78bfa;
}

.empty-state, .state-wrap {
  text-align: center;
  color: var(--text-muted);
  font-size: 14px;
}
</style>
/* ===== src/views/HistoryView.vue ===== */
<template>
  <div>
    <h1 class="page-title">历史数据</h1>
    <p class="page-subtitle">历年台风统计与趋势分析</p>

    <div v-if="loading" class="loading-state">加载中...</div>
    <div v-else-if="error" class="state-wrap">
      <ErrorState title="历史统计加载失败" :description="error" @retry="loadStats" />
    </div>

    <template v-else>
      <!-- Yearly Summary -->
      <div class="stats-grid">
        <div class="stat-card glass-card">
          <div class="ss-label">历史台风总数</div>
          <div class="ss-value">{{ typhoonStore.historicalTyphoons.length }}</div>
          <div class="ss-sub">历年累计</div>
        </div>
        <div class="stat-card glass-card">
          <div class="ss-label">超强台风数</div>
          <div class="ss-value red">{{ superTyphoonCount }}</div>
          <div class="ss-sub">占比 {{ superTyphoonPercent }}%</div>
        </div>
        <div class="stat-card glass-card">
          <div class="ss-label">登陆台风数</div>
          <div class="ss-value orange">{{ landedCount }}</div>
          <div class="ss-sub">影响沿海地区</div>
```

```
        </div>
        <div class="stat-card glass-card">
          <div class="ss-label">平均最大风速</div>
          <div class="ss-value blue">{{ avgWindSpeed }} <span style="font-size:14px;font-weight:400;">m/s</span></div>
          <div class="ss-sub">历史统计</div>
        </div>
      </div>

      <!-- Charts -->
      <div class="content-grid" style="margin-bottom: 28px;">
        <Charts title="台风强度分布" :option="yearPieOption" :height="320" />
        <Charts title="历年台风数量趋势" :option="trendOption" :height="320" />
      </div>

      <!-- Historical Typhoon Table -->
      <TyphoonTable title="历史台风记录" :typhoons="typhoonStore.historicalTyphoons" :showMore="false" />
    </template>
  </div>
</template>

<script setup>
import { computed, onMounted, ref } from 'vue'
import Charts from '../components/Charts.vue'
import TyphoonTable from '../components/TyphoonTable.vue'
import ErrorState from '../components/ErrorState.vue'
import { useTyphoonStore } from '../stores/typhoon'
import { http } from '../utils/request'
import { levelColor } from '../constants/typhoon'

const typhoonStore = useTyphoonStore()

const loading = ref(false)
const error = ref('')
const stats = ref({ levelDistribution: [], yearTrend: [] })

const historicalTyphoons = computed(() => typhoonStore.historicalTyphoons)
const superTyphoonCount = computed(() =>
  historicalTyphoons.value.filter((t) => t.level === '超强台风' || t.level === '强台风').length
)
const superTyphoonPercent = computed(() =>
  historicalTyphoons.value.length
    ? Math.round((superTyphoonCount.value / historicalTyphoons.value.length) * 100)
    : 0
)
const landedCount = computed(() =>
  historicalTyphoons.value.filter((t) => (t.affectedCities || []).length > 2).length
)
const avgWindSpeed = computed(() => {
  if (!historicalTyphoons.value.length) return 0
  const sum = historicalTyphoons.value.reduce((s, t) => s + (t.windSpeed || 0), 0)
```

```
  return Math.round(sum / historicalTyphoons.value.length)
})

async function loadStats() {
  loading.value = true
  error.value = ''
  try {
    const data = await http.get('/statistics')
    stats.value = { ...stats.value, ...(data || {}) }
  } catch (e) {
    if (!e?.isUnauthorized) error.value = e?.message || '历史统计加载失败'
  } finally {
    loading.value = false
  }
}

const yearPieOption = computed(() => ({
  tooltip: { trigger: 'item' },
  series: [
    {
      type: 'pie',
      radius: ['50%', '75%'],
      center: ['50%', '55%'],
      roseType: 'area',
      itemStyle: { borderRadius: 4, borderColor: '#0f1923', borderWidth: 2 },
      label: { color: '#8899aa', fontSize: 12 },
      data: (stats.value.levelDistribution || []).map((d) => ({
        ...d,
        itemStyle: { color: levelColor(d.name) }
      }))
    }
  ]
}))

const trendOption = computed(() => {
  const trend = stats.value.yearTrend || []
  const years = trend.map((y) => y.year)
  return {
    tooltip: { trigger: 'axis' },
    legend: { data: ['台风总数', '超强/强台风'], textStyle: { color: '#8899aa' }, top: 0 },
    grid: { left: '3%', right: '4%', bottom: '3%', top: 36, containLabel: true },
    xAxis: {
      type: 'category',
      data: years,
      axisLine: { lineStyle: { color: 'rgba(255,255,255,0.08)' } },
      axisLabel: { color: '#8899aa' }
    },
    yAxis: {
      type: 'value',
      splitLine: { lineStyle: { color: 'rgba(255,255,255,0.04)' } },
```

```
      axisLabel: { color: '#8899aa' }
    },
    series: [
      {
        name: '台风总数',
        type: 'line',
        data: trend.map((y) => y.total),
        smooth: true,
        symbol: 'circle',
        symbolSize: 8,
        lineStyle: { color: '#3b82f6', width: 3 },
        itemStyle: { color: '#3b82f6' },
        areaStyle: {
          color: {
            type: 'linear', x: 0, y: 0, x2: 0, y2: 1,
            colorStops: [
              { offset: 0, color: 'rgba(59,130,246,0.2)' },
              { offset: 1, color: 'rgba(59,130,246,0.02)' }
            ]
          }
        }
      },
      {
        name: '超强/强台风',
        type: 'line',
        data: trend.map((y) => y.super),
        smooth: true,
        symbol: 'diamond',
        symbolSize: 8,
        lineStyle: { color: '#ef4444', width: 2, type: 'dashed' },
        itemStyle: { color: '#ef4444' }
      }
    ]
  }
})

onMounted(loadStats)
</script>

<style scoped>
.stat-card {
  padding: 24px; text-align: center;
}

.ss-label {
  font-size: 12px; color: var(--text-muted);
  text-transform: uppercase; letter-spacing: 0.5px; margin-bottom: 8px;
}

.ss-value {
```

```
  font-size: 36px; font-weight: 700; letter-spacing: -1px; margin-bottom: 4px;
}

.ss-value.red { color: var(--red); }
.ss-value.orange { color: var(--orange); }
.ss-value.blue { color: var(--blue); }

.ss-sub {
  font-size: 12px; color: var(--text-muted);
}

.loading-state, .state-wrap {
  text-align: center; color: var(--text-muted); font-size: 14px; padding: 60px 0;
}
</style>
/* ===== src/views/NotFound.vue ===== */
<template>
  <div class="nf-page">
    <div class="nf-card glass-card">
      <div class="nf-code">404</div>
      <h1 class="nf-title">页面不存在</h1>
      <p class="nf-desc">
        你访问的路径 <code class="nf-path">{{ path }}</code> 没有对应的页面。
      </p>
      <div class="nf-actions">
        <router-link to="/" class="btn btn-primary">返回仪表盘</router-link>
        <button class="btn" type="button" @click="$router.back()">返回上一页</button>
      </div>
    </div>
  </div>
</template>

<script setup>
import { computed } from 'vue'
import { useRoute } from 'vue-router'

const route = useRoute()
const path = computed(() => route.fullPath)
</script>

<style scoped>
.nf-page {
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 32px;
  background: var(--bg-primary);
}

```

```
.nf-card {
  width: 100%;
  max-width: 460px;
  padding: 40px;
  text-align: center;
}

.nf-code {
  font-size: 56px;
  font-weight: 700;
  letter-spacing: -2px;
  color: var(--blue);
  line-height: 1;
  margin-bottom: 12px;
}

.nf-title {
  font-size: 20px;
  font-weight: 700;
  margin-bottom: 8px;
}

.nf-desc {
  font-size: 13px;
  color: var(--text-secondary);
  margin-bottom: 24px;
  line-height: 1.6;
}

.nf-path {
  background: rgba(255, 255, 255, 0.06);
  padding: 2px 8px;
  border-radius: 4px;
  font-family: 'JetBrains Mono', Consolas, monospace;
  font-size: 12px;
  color: var(--text-primary);
  word-break: break-all;
}

.nf-actions {
  display: flex;
  gap: 12px;
  justify-content: center;
  flex-wrap: wrap;
}

.nf-actions .btn {
  text-decoration: none;
}
</style>
```